Neurocomputing 680 (2026) 133234

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

SaBER-LLMs: Security-aware behavior embedding representation via large language models for fine-grained malware analysis

Chao Jing[a, b], Yun Wu[a, b], Chaoyuan Cui[a, b,*]

[a] Institute of Intelligent Machines, Hefei Institutes of Physical Sciences, Chinese Academy of Sciences, Hefei, 230031, PR China
[b] University of Science and Technology of China, Hefei, 230026, PR China

HIGHLIGHTS

- An LLM-based malware detection framework with expert analysis and semantic perception.
- Two-stage prompting converts API triplets into security-aware text and sensitivity labels.
- Sensitivity-guided compression reduces state explosion and extracts key behavioral snippets.
- Extensive experiments demonstrate higher accuracy, lower latency, and strong robustness to adversarial attacks.

ARTICLE INFO

Communicated by Z. Xu

Keywords:
Malware detection
API call sequence
Large language model
Prompt engineering
Embedding learning

ABSTRACT

Increasing API call sequence-based malware detection methods attempt to incorporate fine-grained semantic information, i.e., parameters and return values, but face two core challenges: *semantic complexity* and *state explosion*, which stem from the difficulty in modeling precise security-aware semantics for domain-specific parameters and return values and the exponential growth of semantic state space. To address these issues, we propose a novel paradigm, *SaBER-LLMs*, which endows large language models with three vital roles: *security experts*, *sensitivity assessors*, and *dual-view semantic perceptors* to leverage their strong semantic understanding and reasoning capabilities, fulfilled through three core components: (i) *Dual-stage prompt engineering* translates low-level behavior triplets into concise, human-readable, security-aware descriptions and assigns them interpretable risk levels. (ii) *Two-stage compression strategy* extracts high-risk behaviors to effectively mitigate state explosion. (iii) *Dual-view combined embedding* encodes both atomic intent and contextual semantics for accurate and resilient detection. Extensive experiments show the superiority of *SaBER-LLMs* in detection performance, interpretability, and robustness against three typical adversarial attacks, i.e., injection, substitution, and parameter-level perturbation, outperforming existing state-of-the-art methods. Specifically, *SaBER-LLMs* achieves 97.71% and 96.43% accuracy over two baselines, reduces downstream classifier inference time, and maintains high defense effectiveness under both black-box and grey-box threat models with impractically large attack overheads, underscoring the innovative paradigm in leveraging LLMs for heightened security efficacy and its practical advantage in real-world threat scenarios.

1. Introduction

The rapid evolution of malware, especially unknown variants, makes effective detection a critical task in cybersecurity defense. Among detection methods, runtime behavior analysis is notably effective for resisting evasion techniques such as code obfuscation and packing, with **Application Programming Interface (API) call sequences** providing an even more precise basis by capturing core interactions between software and the operating system (e.g., file access, registry changes, network activity, and process creation).

Mainstream detection methods based on API sequences, whether using traditional machine learning (e.g., sequence pattern mining, Hidden Markov Models) or deep learning models (e.g., Recurrent Neural Networks (RNN), Long Short-Term Memory networks (LSTM), Transformers), typically focus on the sequences of **API call name**, such as `CreateFile→WriteFile→CloseFile`. While effective to some

* Corresponding author at: Institute of Intelligent Machines, Hefei Institutes of Physical Sciences, Chinese Academy of Sciences, Hefei, 230031, PR China.
Email addresses: jingchao@mail.ustc.edu.cn (C. Jing), wuyun@rntek.cas.cn (Y. Wu), cycui@iim.ac.cn (C. Cui).

https://doi.org/10.1016/j.neucom.2026.133234
Received 18 December 2025; Received in revised form 14 February 2026; Accepted 4 March 2026
Available online 6 March 2026
0925-2312/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

extent, these methods face a fatal limitation: overlooking the fine-grained information within API calls, namely, the **parameters** and **return value**, which are vital for capturing behavioral semantics and accurately assessing intent:

• Parameters convey behavioral intent by specifying targets and operations, e.g., `WriteFile` to a critical system file (`C:\ Windows\ system32\ config\ SAM`) indicates higher risk than writing to a temporary file, and `RegSetValue` modifying an auto-start registry key is more suspicious than one altering benign settings.

• Return value reveals execution results and contextual semantics, e.g., `CreateFile` returning `STATUS_SUCCESS` differs significantly from `STATUS_ACCESS_DENIED`, and `FindFirstFile` returning `INVALID_HANDLE_VALUE` may imply probing behavior, all of which expose system state and potential attack intent.

Unlike the limited and discrete space of API names, the state space formed by parameters and return values is enormous, heterogeneous, and semi-structured, encompassing unbounded domains such as file paths, URLs, and registry keys, which lead to two major challenges:

(*i*) *Semantic complexity.* Such domain-specific parameters and return values make designing static rules or building semantic models extremely difficult to accurately and comprehensively capture their deep security-aware implications.
(*ii*) *State explosion.* Directly modeling such diverse parameters and return values will exponentially expand the semantic space, exacerbating data sparsity, model complexity, and training instability, thus impeding generalization.

A few existing studies attempting to incorporate fine-grained information typically adopt several strategies:

① *Statistics*. Count parameter and return value distributions (e.g., mean, frequency, or entropy) [3,4,8,32,40].
② *Hash encoding.* Map parameters and return values into fixed-size vectors using predefined dictionaries or binary-to-decimal encoding schemes [12,50].
③ *Pretrained model embedding.* Input parameters and return values as plain strings into embedding models (e.g., *Word2Vec* or *BERT*) pretrained on natural language corpora (e.g., Wikipedia, BookCorpus) to obtain embeddings [18,36].

However, ① is highly fragile and vulnerable to API call injection, where attackers can manipulate statistics by injecting no-op but frequent API calls with benign-looking parameters and return values, thereby evading detection [33]. ②, while efficient, suffers from *hash collision*, i.e., different inputs yield identical hashes, reducing discriminative power and hindering security-sensitive distinctions [18]. ③ offers flexibility but exhibits a significant domain knowledge gap in operating system internals and security expertise, e.g., security sensitivity of specific paths or precise meanings of error codes (`0xc000000f` corresponds to `STATUS_NO_SUCH_FILE`), causing semantically distorted embeddings. We posit that addressing the above challenges necessitates an intelligent approach capable of (*i*) deeply understanding the intrinsic security-aware semantics of API calls with their parameters and return values, and (*ii*) effectively compressing the combinatorial state space into a tractable representation.

Novelty. Recent advancements in the Large Language Models (LLMs) have demonstrated unprecedented capabilities in language understanding, knowledge reasoning, and context awareness, which greatly inspire our novel proposal: *Utilizing LLMs to act as "security domain experts", transforming raw, low-level, technical API call sequences (including names, parameters, and return values) into high-level, human-readable, and security-semantic-rich natural language descriptions of behavior, further to act as "sensitivity assessors", compressing them into discrete sensitivity levels, and finally to act as "intra-semantic" and "inter-semantic perceptors", forming a comprehensive security-aware embedding foundation.*

In this paper, we propose **SaBER-LLMs**, **s**ecurity-**a**ware **b**ehavior **e**mbedding representation via **LLMs**, which utilizes the knowledge transfer and context understanding capabilities of LLMs (e.g., GPT-4 [2], DeepSeek [15,27], Qwen[6], LLaMA[41], Claude-3[5], Gemini-1.5[39]) to first translate each API call with its parameters and return value into natural language sentences with explicit security semantics, and then quantize them into sensitivity levels. Further, we propose two-stage compression strategy based on sensitivity levels, reducing the state space of sequence from $\mathcal{O}(M^n)$ to $\mathcal{O}(5^m)$ (M is large, and $m \ll n$), making modeling feasible. Finally, we use Embedding-LLMs to generate complementary *intra-call* and *inter-call* security-aware embeddings from the compressed sequence, and input combined embedding into two baseline classifiers. The contributions are summarized as follows:

- A detection paradigm, *SaBER-LLMs*, using LLMs as *"security experts"*, *"sensitivity assessors"*, and *"intra&inter-semantic perceptors"* for semantic translation, space compression, and representation of fine-grained API sequences, enriches security-aware semantics, reduces combinatorial explosion, and enables effective embedding learning.
- Three pivotal modules are designed: *dual-stage prompt engineering* converts API triplets to security-aware descriptions and sensitivity levels, *two-stage compression* markedly reduces state complexity, and *dual-view combined embedding* captures both atomic and contextual security semantics.
- Extensive experiments are conducted on large-scale public datasets, demonstrating the effectiveness of *SaBER-LLMs*, which achieves significant improvements in detection performance and robustness against adversarial samples compared to existing state-of-the-art (SOTA) methods.

2. Motivation and preliminaries

2.1. Motivation

Observation 1. *An individual API call may not show malicious intent, but when combined with specific parameters and return values, it can reveal malicious behavior patterns [18].* This insight motivates our investigation into deep analysis of API calls with their parameters and return values to determine the precise security implications of software behavior. Here are some representative API calls observed in malware execution traces, performing three types of manipulations:

• **File system operations:**
① `NtCreateFile("C:\windows\SysWOW64\WindowsPowerShell\v1.0\powershell_ise.exe")→c000003a`
② `WritePEFile("C:\program\icUC.exe")→0`
③ `NtReadFile("NamedPipe\wkssvc")→c000014b`
④ `DeleteFileW("C:\program\1.exe.dat")→0`
• **Registry modifications:**
⑤ `NtSetValueKey("HKEY_LOCAL_MACHINE\SOFTWARE\BNJWLXHMPC", "Start")→0`
⑥ `NtSetValueKey("HKEY_LOCAL_MACHINE\SOFTWARE\MICROSOFT\SECURITY CENTER\SVC", "AntiVirusOverride")→c0000022`
⑦ `NtDeleteKey("HKEY_LOCAL_MACHINE\SOFTWARE\JYTTNKMDLF\SECURITY")→0`
• **Process management:**
⑧ `CreateProcessInternalW("c:\ceopb0.exe")→1`
⑨ `NtTerminateProcess("bg.exe", 35)→abadbeef`

The inclusion of parameters and return values significantly enriches the semantics beyond API names alone, such as target files, registry keys, and status codes. Yet the critical security narratives concealed in details, especially attack intent and system impact, remain inscrutable to non-experts.

Observation 2. *Extracting potential and genuine security intelligence from fine-grained information requires specialized expertise that transcends surface-level observation [45].* While non-specialists may recognize some file operations or registry modifications as distinct from routine operations, only security domain experts can interpret their true implications:

- `WindowsPowerShell` access patterns indicate *living-off-the-land binary (LOLBin) exploitation potential.*
- `AntiVirusOverride` modifications represent *deliberate attempts to disable security monitoring.*
- Registry keys like `BNJWLXHMPC` reveal *anti-detection* through obfuscated persistence mechanisms.
- `0xabadbeef` is a non-standard return value indicating *invalid execution* or *willfully obscured control flow.*

This semantic gap requires professional domain knowledge for accurate threat assessment. However, the enormous malware traces, exceeding even millions of API calls, make comprehensive expert-driven analysis prohibitively costly and impractical for real-world threat intelligence.

Inspiration 1. *LLMs show exceptional aptitude as automated security experts, capable of bridging the semantic gap between raw API calls and actionable threat intelligence through their strong security knowledge reserve and reasoning capabilities.*

The paradigm shift from raw API calls to security-aware semantic descriptions fundamentally alleviates the interpretability and scalability flaws of current malware analysis.

Observation 3. *Malicious attacks emerge not from isolated API calls but from their coordinated execution patterns, combining varying sensitivity-level operations [26].* Here is a malicious spread and self-startup execution snippet [18]:

• Dissemination and self-startup:
① `GetModuleFileName(NULL, "UuYUIQAM.exe", 0x32)→"C:\Documents and Settings\All Users\pAgIwsIY\UuYUIQAM.exe"`
(get the current binary path)
② `CopyFileExW("C:\Documents and Settings\All Users\pAgIwsIY\UuYUIQAM.exe", "C:\WINDOWS\system32\iwkg.exe", False)→1`
(overwrite system file)
③ `SetFileHiddenOrReadOnly("C:\WINDOWS\system32\iwkg.exe", 00000087)→0` (hidden file)
④ `Process32FirstW()→1` (enumerate process)
⑤ `CreateProcessInternalW("C:\WINDOWS\system32\iwkg.exe")→1` (start the process)
⑥ `NtSetValueKey("HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\Current Version\Run","iwkg.exe")→0`
(achieve self-startup)

It exemplifies the sensitivity-stratified attack pattern: ① and ④ appear harmless (querying executable path, enumerating processes) but provide essential prerequisites. ⑤ is suspicious when targeting system executables. ②, ③, and ⑥ are highly sensitive (system file overwrite, attribute hiding, and persistence setup). This multi-sensitivity-level coordinated use forms an entire malicious routine, showing the security risks arising from contextual API call interactions rather than isolated calls. Hence, it's essential to capture security-aware semantics not only within, but also between fine-grained API calls. However, the vast and heterogeneous API call semantic space exponentially expands the combinatorial behavior pattern space, hindering the convergence and generalization of detection models. Fortunately, malware behavior has *Locality*:

Observation 4. *Malicious attacks are usually implemented through a small number of API calls (maybe several or dozens), as opposed to flooding the whole sequence. [18].*

Before analysing inter-call behavior patterns, if we can (*i*) compress the fine-grained API call semantic space and (*i*) distill the effective behavior subsequences, the convergence and efficiency of detection models will be significantly improved. LLMs with their strong inductive capabilities, offer a promising pathway to achieve both objectives.

Inspiration 2. *LLMs act as API sensitivity assessors grounded in prior security-aware intelligence, enabling the compression of state space and refinement of key behavioral snippets.*

This sensitivity-guided condensation of behavior sequences significantly improves model convergence and generalization, addressing the inefficiency caused by excessive semantic redundancy in large-scale API call traces.

2.2. Preliminaries

What are LLMs? LLMs are transformer-based deep neural networks with billions of parameters, pre-trained on massive text corpora to perform many natural language tasks, capable of understanding and generating human-like text, and enabling downstream applications with minimal task-specific tuning. Their core capabilities derive from:

- *Scale:* Model size (>0.5B parameters) and training data (>1T tokens) enable emergent abilities.
- *Architecture:* Multi-head attention mechanisms in Transformers [43] capture long-range dependencies.
- *Pre-training objective:* Masked language modeling (BERT) and autoregressive prediction (e.g., GPT-4).

In recent years, many LLMs have emerged from academia and industry. Table 1 lists some representative LLMs.

How to use LLMs? *Prompt engineering* is a common technique to guide LLMs by carefully crafted prompts, typically consisting of task instructions, optional input-output examples, and the target query, which has three main strategies:

- *Zero-shot prompting:* Directly ask LLMs to perform a task without examples.
- *Few-shot prompting:* Provide annotated examples to guide the desired output style or reasoning.
- *Chain-of-thought prompting:* Explicitly encourage LLMs to "step-by-step" reasoning.

In this work, we design two-stage structured prompts that convert fine-grained APIs into natural language descriptions and sensitivity levels enriched with security semantics.

3. Methodology

3.1. Approach overview

To enable fine-grained, interpretable, and efficient malware behavior analysis, we propose a novel framework that leverages LLMs for semantic translation, sensitivity quantification, and behavior embedding. For clarity, the main notation definitions are shown in Table 2.

Given x, dynamically extracted as $\mathbf{S}$ by monitoring in a virtual sandbox (i.e., $x \longmapsto \mathbf{S}$). Each $s \in \mathbf{S}$ contains zero or multiple parameters and a return value, represented as a basic behavior triplet $[API, [P_1, \ldots, P_k], RV]$. The overview architecture and implementation

Table 1
Representative LLMs and their architectures.

Model	Organization	Parameters	Context
GPT-4[2]	OpenAI	~1.8T[a]	128K
DeepSeek-V3[27]	DeepSeek	671B	128K
DeepSeek-R1[15]	DeepSeek	1.5~671B	128K
Qwen-3[46]	Alibaba	0.6~235B	32~128K
LLaMA-3[14]	Meta	405B	128K
Claude-3[5]	Anthropic	200B	200K
Gemini-1.5[39]	Google	32~200B[a]	1M

[a] Estimated via model parallelism.

Table 2
Main Notation Definitions.

Term	Definition	Explanation
x	—	Software sample
s	$[API, [P_1, \dots, P_k], RV]$[a]	Basic behavior triplet
$\mathbf{S}$	$[s_1, s_2, \dots, s_n]$	Sequence of n calls s_i
$\mathcal{S}$	$\{\mathbf{S}_1, \mathbf{S}_2, \dots, \mathbf{S}_N\}$	Set of API sequences
t	—	Security description
$\mathbf{T}$	$[t_1, t_2, \dots, t_n]$	Sequence of n texts t_i
Ω_p	$\mathbf{S} \mapsto \mathbf{T}$	Map: $\mathbf{S}$ to $\mathbf{T}$
l_{max}	—	Max text length
y^{sl}	—	Sensitivity level
Y^{sl}	{B., L., M., H., C.}[b]	Five sensitivity levels
$\mathbf{Y}^{sl}$	$[y_1^{sl}, y_2^{sl}, \dots, y_n^{sl}]$	Sequence of n levels y_i^{sl}
Ω_s	$\mathbf{S}\&\mathbf{T} \mapsto \mathbf{Y}^{sl}$	Map: $(\mathbf{S}, \mathbf{T})$ to $\mathbf{Y}^{sl}$
s	$[API, y^{sl}]$	s labeled with y^{sl}
$\hat{\mathbf{S}}$	$[\hat{s}_1, \hat{s}_2, \dots, \hat{s}_n]$	Sequence of n calls $\hat{s}_i$
τ	—	Level threshold
$\hat{\mathbf{S}}_{\text{filtered}}$	$[\hat{s}_i \mid y_i^{sl} \geq \tau, i = 1, \dots, n]$	Calls with level $\geq \tau$
Ω_e	$\mathbf{T} \mapsto \boldsymbol{E}^{ita}$	Map: $\mathbf{T}$ to $\boldsymbol{E}^{ita}$
$\boldsymbol{H}$	—	Token-level embedding
Ω_e^{token}	$\hat{\mathbf{S}} \mapsto \boldsymbol{H}$	Map: $\hat{\mathbf{S}}$ to $\boldsymbol{H}$
$\boldsymbol{E}^{ita}$	$[e^a_{i_1} e^a_{i_2} \cdots e^a_{i_m}]$	Intra-call embedding
$\boldsymbol{E}^{itr}$	$[e^r_{i_1} e^r_{i_2} \cdots e^r_{i_m}]$	Inter-call embedding
$\boldsymbol{E}^{+}$	$[e^r_{i_1} \| e^a_{i_1}, \dots, e^r_{i_m} \| e^a_{i_m}]$	Combined embedding

[a] P_i refers to the i-th parameter and RV means the return value.
[b] The abbreviations of the five sensitivity levels.

process of *SaBER-LLMs* are shown in Fig. 1 and Algorithm 1, which operate in a multi-stage pipeline, as illustrated below:

Step 1 (lines 2~4): prompt-based semantic translation. Design a structured prompt Ω_p to translate each API s_i into a human-readable, security-aware description $t_i \in \mathbf{T}$ using LLMs, crafted with domain-priming, structured rules, and semantic distillation to ensure consistency and interpretability.

Step 2 (lines 5~7): prompt-based sensitivity rating. Design a second-stage prompt Ω_s with s_i and the generated text t_i to assign each behavior a sensitivity level y_i^{sl} from a five-grade taxonomy, which enhances threat interpretability and enables quantitative risk abstraction.

Step 3 (lines 8~11): two-stage state space compression. To manage the high-dimensional semantic space, we compress s_i into $\hat{s}_i = [API, y_i^{sl}]$, reducing the state space from $\mathcal{O}(M)$ to $\mathcal{O}(1)$. Furthermore, filtering based on a threshold τ extracts the high-risk subsequence $\hat{\mathbf{S}}_{\text{filtered}}$, preserving temporal order while discarding noisy or harmless behaviors.

Step 4 (lines 12~19): dual-view embeddings Ω_e&Ω_e^{token}. We extract two complementary embeddings based on $\hat{\mathbf{S}}_{\text{filtered}}$:

- *Intra-call embedding* $\boldsymbol{E}^{ita}$: Derived from t_i to capture fine-grained semantic intent of individual behaviors.
- *Inter-call embedding* $\boldsymbol{E}^{itr}$: Captures relational context among calls by encoding the full sequence composed of $[\texttt{API}_i + y_i^{sl}]$ into LLM token-level representations.

Algorithm 1: The Training of *SaBER-LLMs*.

Input: The training set $\{(\mathbf{S}_i, y_i)\}_{i=1}^{|S_{train}|}$;
The max length of output l_{max};
The sensitivity threshold τ;
The LLMs Ω_p, Ω_s;
The Embedding-LLMs Ω_e, Ω_e^{token};
The classifier set $C = \{C_1, C_2\}$.
Output: The detection model *SaBER-LLMs*.

1 **foreach** $\mathbf{S}_i \in S_{train}$ **do**
  // Prompt-based semantic translation.
2 **foreach** $s_{i,j} = [API_j, [P_{1_j}, \dots, P_{k_j}], RV_j] \in \mathbf{S}_i$ **do**
3 Query LLMs with $\Omega_p(s_{i,j})$;
4 Receive description $t_{i,j} \in \mathbf{T}_i$;
  // Prompt-based sensitivity rating.
5 **foreach** $(s_{i,j}, t_{i,j})$ *pair* **do**
6 Query LLMs with $\Omega_s(s_{i,j}, t_{i,j})$;
7 Obtain sensitivity level $y_j^{sl} \in Y^{sl}$;
  // Two-stage state space compression.
8 **foreach** $s_{i,j} \in \mathbf{S}_i$ **do**
9 Convert to labeled call $\hat{s}_{i,j} = [API_j, y_j^{sl}]$;
10 Form compressed sequence $\hat{\mathbf{S}}_i = [\hat{s}_{i,1}, \dots, \hat{s}_{i,n_i}]$;
11 Filter sequence by τ to obtain $\hat{\mathbf{S}}_{i,\text{filtered}}$;
  // Generate dual-view embeddings.
12 **foreach** $\hat{s}_{i,j_k} \in \hat{\mathbf{S}}_{i,filtered}$ **do**
13 Extract intra-call embedding $e^a_{i,j_k} = \Omega_e(t_{i,j_k})$;
14 Construct $\boldsymbol{E}_i^{ita} = [e^a_{i,j_1}, \dots, e^a_{i,j_{m_i}}]$;
15 Construct paragraph P_i by concatenating $\hat{s}_{i,j}$.
16 Tokenize P_i using Ω_e^{token} to obtain $\boldsymbol{H}_i$;
17 **foreach** $\hat{s}_{i,j_k} \in \hat{\mathbf{S}}_{i,filtered}$ **do**
18 Average token span for $\hat{s}_{i,j_k}$ to obtain e^r_{i,j_k};
19 Construct $\boldsymbol{E}_i^{itr} = [e^r_{i,j_1}, \dots, e^r_{i,j_{m_i}}]$;
  // Fuse embeddings.
20 Concatenate embedding $\boldsymbol{E}_i^{+} = [e^r_1 \| e^a_1, \cdots, e^r_m \| e^a_m]$;
 // Training classifiers.
21 **foreach** *classifier* $C_k \in C$ **do**
22 Train C_k using $\{\boldsymbol{E}_i^{+}, y_i\}_{i=1}^{|S_{train}|}$;
23 Evaluate C_k on validation set;
24 Save the best-performing model;

Step 5 (lines 20~24): embedding combination and detection. The intra & inter-call embeddings are concatenated as $\boldsymbol{E}^{+}$, forming a rich security representation, and are input to two baseline classifiers to evaluate the effectiveness of $\boldsymbol{E}^{+}$.

This LLM-enhanced pipeline not only provides semantic transparency and dynamic risk modeling for each API operation, but also

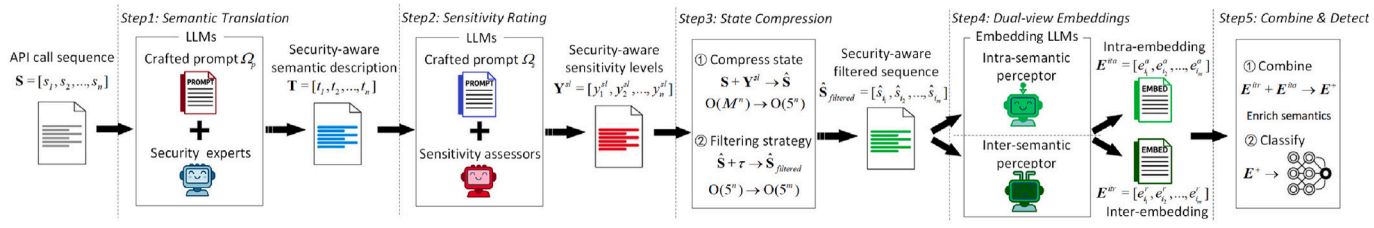


Fig. 1. The overview architecture of *SaBER-LLMs* consists of five steps, which translate raw API triplets into concise, human-readable security descriptions, assign interpretable sensitivity labels, filter low-risk behaviors to compress space, and extract dual-view embeddings combining intent-level and context-aware semantics, semantically enhancing low-level API behavior sequences by leveraging LLMs. A detailed case analysis is in Section 4.1.

achieves sequence compression and representation learning, forming a unified and explainable malware analysis framework.

3.2. Prompt engineering for security descriptions

To utilize LLMs for behavior interpretation, we first construct a structured prompt to convert the fine-grained behavior triplet $s = [API, [P_1, \ldots, P_k], RV]$ into a human-readable, security-aware semantic description, exploiting LLMs' strong knowledge base and contextual reasoning to decode the functionality and potential intent behind each API unit. Our prompt adheres to three core design principles:

① *Identity priming.* Assigning LLMs specific operational identities enhances reasoning depth and output quality. Thus, we prime LLMs as seasoned Windows security analysts to activate domain expertise [47].

② *Structured rules.* Enforcing strict I/O formats eliminates hallucinations and parsing errors, e.g, requiring plain-text output while prohibiting special symbols (e.g., \ n, \ t) and non-natural formats (e.g., Markdown or JSON) [45].

③ *Semantic distillation.* Overly long descriptions will increase the time consumption and computing space. Word limit can force abstraction of security essence, so we explicitly set output less than l_{max} words for each triplet [45].

Inspired by the above, we design a concise yet expressive prompt engineering mapping Ω_p, formatted as follows:

> `[System]`: You are a seasoned Windows security analyst. Given the API behavior triplet: `[Input]`: API:{name}, Params:{[p1,…,pn]}, Return:{val}
> `[Limit]`: Provide a brief explanation (< l_{max} words) of what the operation likely represents or its context. Only output each explanation in the following format:
> `[Output]`: <natural language description>
> `[Rule]`: Do not output anything else, e.g., special symbols and non-natural formats.

Through Ω_p, each triplet $s_i \in \mathbf{S}$ generates security-aware descriptive text $t_i \in \mathbf{T}$, i.e., $t_i = \Omega_p(s_i)$. Further, the security intelligence of software sample x, namely, $\mathbf{T}_x$ is obtained:

$$\mathbf{T}_x = [t_1, t_2, \ldots, t_n] = [\Omega_p(s_1), \Omega_p(s_2), \ldots, \Omega_p(s_n)]. \quad (1)$$

This semantic transformation can significantly improve the interpretability of API calls and lay the foundation for subsequent security-aware analysis.

3.3. Prompt engineering for sensitivity levels

To assess the potential threat of s_i, we define five sensitivity levels in Table 3, considering its parameters, return value and latent security-aware semantics, and further prompt LLMs to evaluate its corresponding sensitivity y_i^{sl}.

Unlike naïve rule-based heuristics, our framework integrates LLMs' reasoning to reflect the combined effect of low-level operations and their latent intent. The mapping Ω_s is designed incorporating t_i generated by Eq. (1) alongside s_i:

> `[System]`: You are a seasoned Windows security analyst and need to evaluate the sensitivity of API behavior triplets based on five-point rating criteria: {Sensitivity & Rating Criteria}.
> `[Input]`: Given the API behavior triplet: API: {name}, Params: {[p1,…,pn]}, Return: {val}, and its security description {t}.
> `[Limit]`: Only return the rating level in this format:
> `[Output]`: <sensitivity>
> `[Rule]`: Do not output anything else than sensitivity.

Table 3
Definition of five sensitivity levels.

Sensitivity	Rating Criteria[a]
Benign	Completely normal behavior. The invoked API is commonly used in system or user-level operations. Parameters are not suspicious and the return value is normal. E.g., reading configuration files, standard GUI actions.
Low	Generally legitimate, but with slightly sensitive targets or parameters. E.g., accessing files in system directories, querying basic user info, or writing to non-critical registry keys, which may be abused, but rarely indicate malicious.
Medium	Potentially suspicious or commonly abused behavior, needing further context. E.g., writing startup registry entries, or creating persistence shortcuts.
High	Highly sensitive behavior typically used in key stages of attacks. E.g., remote thread injection, allocation of executable memory, modifying sensitive registry keys like Run or Winlogon, or attempting privilege escalation.
Critical	Extremely suspicious and strongly malicious. Rare in normal applications. E.g., successful code injection, driver installation, disabling security tools, token manipulation, oranti-sandbox/anti-debugging techniques.

[a] The five levels align with risk categorization frameworks such as NIST SP 800–30 (Very Low/Low/Moderate/High/Very High) [34].

Through Ω_s, each triplet $s_i \in \mathbf{S}$ combined with $t_i \in \mathbf{T}$ generates its sensitivity level $y_i^{sl} \in Y_i^{sl}$, i.e., $y_i^{sl} = \Omega_s(s_i, t_i)$. Further, the behavioral sensitivity of software sample x, namely, $\mathbf{Y}_x^{sl}$ is obtained:

$$\mathbf{Y}_x^{sl} = [y_1^{sl}, \ldots, y_n^{sl}] = [\Omega_s(s_1, t_1), \ldots, \Omega_s(s_n, t_n)]. \quad (2)$$

This dual-stage prompt not only captures nuanced security semantics but also enables dynamic risk quantification, which is essential for fine-grained behavioral threat modeling.

3.4. Two-stage state space compression

Building upon $y^{sl} \in Y^{sl}$, we propose a two-stage compression framework to address the combinatorial explosion of API sequence patterns. The core insight is:

3.4.1. Compress semantics into sensitivity space

We convert each triplet s into $\hat{s}$ labeled by its sensitivity level y^{sl}, i.e.,

$$s = [API, [P_1, \ldots, P_k], RV] \rightarrow \hat{s} = [API, y^{sl}], \quad (3)$$

so the raw API call sequence $\mathbf{S}$ will be transformed into $\hat{\mathbf{S}}$:

$$\mathbf{S} = [s_1, s_2, \ldots, s_n] \rightarrow \hat{\mathbf{S}} = [\hat{s_1}, \hat{s_2}, \ldots, \hat{s_n}]. \quad (4)$$

Suppose s has M possible states (potentially very large or even unbounded). After compression, s is mapped to $\hat{s}$, one of only five sensitivity levels, whose state space is reduced from $\mathcal{O}(M)$ to $\mathcal{O}(1)$. For $\mathbf{S}$ with $\mathcal{O}(M^n)$ states, the space of $\hat{\mathbf{S}}$ is compressed to $\mathcal{O}(5^n)$, which yields an exponential compression effect on complexity.

3.4.2. Filter sequences based on sensitivity levels

Inspired by *observation 4*, we design a *sensitivity level-based filtering strategy* to extract effective behavior snippets. Specifically, we define a sensitivity threshold $\tau \in Y^{sl}$ (e.g., $\tau = Medium$, which retains $\hat{s}$ rated as τ or higher) and filter out all $\hat{s}$ below τ:

$$\hat{\mathbf{S}}_{\text{filtered}} = [\hat{s}_i \mid y_i^{sl} \geq \tau, \text{for } i = 1, \ldots, n], \quad (5)$$

where the original temporal order is still preserved. It transforms $\hat{\mathbf{S}}$ into $\hat{\mathbf{S}}_{\text{filtered}} = [\hat{s}_{i_1}, \ldots, \hat{s}_{i_m}]$, whose length is reduced from n to m ($m \ll n$), also achieving significant compression.

Empirical analysis shows that both benign and malicious samples contain a certain proportion of harmless and non-informative APIs that

can be removed without affecting key behavioral semantics. Sensitivity-based filtering can shorten API sequences, reduce model complexity, remove behavioral noise, and sharpen focus on attack-relevant behaviors.

3.5. Obtain intra-call and inter-call embedding

Based on $\hat{\mathbf{S}}_{\text{filtered}}$, we obtain two complementary security-aware embeddings by LLMs, i.e., intra-call and inter-call embeddings, which capture semantic details and contextual relationships:

3.5.1. Intra-call embedding via security descriptions

For $\hat{s}_{i_k} \in \hat{\mathbf{S}}_{\text{filtered}}$, we can extract its *intra-call embedding* $e^a_{i_k}$ [1] by inputting its corresponding t_{i_k} generated in Section 3.2 into Embedding-LLMs (denoted as Ω_e):

$$e^a_{i_k} = \Omega_e(t_{i_k}) \in \mathbb{R}^{d^a}, \tag{6}$$

where d^a denotes its embedding dimension. Thus, the **intra**-call embedding matrix $\boldsymbol{E}^{ita}$ for $\hat{\mathbf{S}}_{\text{filtered}}$ is:

$$\boldsymbol{E}^{ita} = \left[e^a_{i_1}, \ldots, e^a_{i_m}\right] = \left[\Omega_e(t_{i_1}), \ldots, \Omega_e(t_{i_m})\right] \in \mathbb{R}^{m \times d^a}, \tag{7}$$

which emphasizes the individual behavior semantics with sensitive intent, abstracted via human-readable descriptions.

3.5.2. Inter-call embedding via context modeling

For $\hat{s}_{i_k} \in \hat{\mathbf{S}}_{\text{filtered}}$, we can generate *inter-call embeddings* $e^r_{i_k}$, modeling its security-aware semantic relationship in $\hat{\mathbf{S}}_{\text{filtered}}$. We first treat $\hat{s}_i$ as a sentence like [*"API + sensitivity."*], e.g., `[NtSetValueKey High.]`, and construct the context paragraph P of $\hat{\mathbf{S}}_{\text{filtered}}$ by concatenating sentences as:

$$P = \oplus_{k=1}^{m} \left(\text{API}_{i_k} \oplus y^{sl}_{i_k}\cdot\right). \tag{8}$$

Then, P is entered to Embedding-LLMs to get *token-level embeddings* (provided by certain LLMs, denoted as Ω^{token}_e):

$$\boldsymbol{H} = \Omega^{token}_e(P) \in \mathbb{R}^{L \times d^r}, \tag{9}$$

where L is the token length of P and d^r is the embedding dimension. For each sentence $\hat{s}_i$, we extract its token index span $[l^{st}_i, l^{ed}_i]$ from *tokenizer(P)*, and average the corresponding embeddings to get e^r_i:

$$e^r_i = \frac{1}{l^{ed}_i - l^{st}_i + 1} \sum_{j=l^{st}_i}^{l^{ed}_i} \boldsymbol{H}_j \in \mathbb{R}^{d^r}.^{1} \tag{10}$$

Finally, the **inter**-call embedding matrix $\boldsymbol{E}^{itr}$ for $\hat{\mathbf{S}}_{\text{filtered}}$ is:

$$\boldsymbol{E}^{itr} = [e^r_{i_1}, e^r_{i_2}, \ldots, e^r_{i_m}] \in \mathbb{R}^{m \times d^r}, \tag{11}$$

capturing relational semantics and structural intent across the sequence, such as suspicious dependency.

3.6. Embedding-combined malware detection

We fuse $\boldsymbol{E}^{ita}$ into $\boldsymbol{E}^{itr}$ as the combined embedding $\boldsymbol{E}^{+}$:

$$\boldsymbol{E}^{+} = \boldsymbol{E}^{itr} \oplus \boldsymbol{E}^{ita} = [e^r_{i_1} \| e^a_{i_1}, \cdots, e^r_{i_m} \| e^a_{i_m}] \in \mathbb{R}^{m \times (d^r + d^a)}, \tag{12}$$

where $\boldsymbol{E}^{ita}$ depicts the fine-grained key behavior semantics and $\boldsymbol{E}^{itr}$ focuses on the relevance between sensitivity-labeled APIs. The dual-view embedding forms a comprehensive security-aware embedding foundation, which can be used for downstream classification. We utilize two representative deep learning baseline networks, allowing variable-length input, i.e., Text-CNN and BiGRU-Attention [31], whose core principles are illustrated in Table 4.

[1] LLMs support shortening dimensions due to *Matryoshka Representation Learning (MRL)*[24] without threatening its representation ability.

Table 4
Two representative state-of-the-art baseline networks.

Networks	Model principles
Text-CNN [21]	$\begin{cases} \boldsymbol{x}_{1:n} = \boldsymbol{x}_1 \oplus \boldsymbol{x}_2 \oplus \cdots \oplus \boldsymbol{x}_n \\ c_i = f(\boldsymbol{w}_c \boldsymbol{x}_{i:i+s-1} + b) \\ \hat{c} = \max\{[c_1, c_2, \cdots, c_{n-s+1}]\} \\ y = \boldsymbol{w}_{\hat{c}}([\hat{c}_1, \ldots, \hat{c}_m] \circ \boldsymbol{r}) + b \end{cases}$ $\oplus$: concatenation; $\boldsymbol{w}$: weight matrix; s: filter size; b: bias term; $\circ$: multiplication; $\boldsymbol{r}$: regularization
BiGRU-Attention [9]	$\begin{cases} r_t = \tanh(\boldsymbol{w}_r[h_{t-1}, x_t] + b_r) \\ u_t = \tanh(\boldsymbol{w}_z[h_{t-1}, x_t] + b_u) \\ \tilde{h}_t = \tanh(\boldsymbol{w}_h[r_t * h_{t-1}, x_t] + b_h) \\ h_t = (1 - u_t) * h_{t-1} + u_t * \tilde{h}_t \\ \alpha_t = \text{softmax}(\boldsymbol{v}^\top \tanh(\boldsymbol{W} h_t + b)) \\ \boldsymbol{c} = \sum_t \alpha_t h_t \end{cases}$ r: reset gate; u: update gate; $\tilde{h}$: update value; α: attention score; c: context vector

4. Understanding SaBER-LLMs

4.1. A step-by-step operational walkthrough

To investigate why *SaBER-LLMs* is effective, we visualize the output step by step with an input behavior snippet in Section 2.1, i.e., $[s_i, \ldots, s_j] \in \mathbf{S}$:

```
ⓘ [GetModuleFileName,[NULL,"UuYUIQAM.exe",0x32],"C:
\Documents and Settings\All Users\pAgIwsIY\UuYUIQAM.exe"]
...
ⓙ [NtSetValueKey,[``HKEY_LOCAL_MACHINE\SOFTWARE\Micro
soft\Windows\Current Version\Run'',``iwkg.exe''],0]
```

• *Step 1:* obtain $[t_i, \ldots, t_j] \in \mathbf{T}$ by Ω_p, as
["Retrieves the full path of the current executable, likely for self-identification or replication purposes.",
…,
"Adds a program to the Windows startup registry, likely to ensure "iwkg.exe" runs automatically when system boots."], translating low-level behaviors into human-readable and concise descriptions enriched with security intent.
• *Step 2:* get $[y^{sl}_i, \ldots, y^{sl}_j] \in \mathbf{Y}^{sl}$ via $\Omega_s\left(s_i, t_i\right)$, namely,

["Low", ⋯, *"Critical"]*,

which enables risk quantification and behavior prioritization through interpretable sensitivity levels.
• *Step 3:* obtain $[\hat{s}_i, \ldots, \hat{s}_j] \in \hat{\mathbf{S}}$ formed of $\hat{s} = [API, y^{sl}]$, i.e.,

[[GetModuleFileName,Low], ⋯, *[NtSetValueKey,Critical]]*,

and extract $\hat{\mathbf{S}}_{\text{filtered}}$ by *filtering strategy* (suppose $\tau = Medium$):
~~[[GetMod..., Low]~~, ⋯, *[NtSetValueKey, Critical]]*
that compresses state space and focuses on high-risk behaviors.
• *Step 4:* feed each t_k of $\hat{s}_k \in \hat{\mathbf{S}}_{\text{filtered}}$ to gain e^a_k by Ω_e, e.g.,

$$e^a_j = \Omega_e(\text{"Adds a ... boots."}) = [0.0336, \ldots, -0.0615] \in \mathbb{R}^{d^a},$$

so $\boldsymbol{E}^{ita} = [\cdots, e^a_j] = [\cdots, [0.0336, \cdots, -0.0615]] \in \mathbb{R}^{|\hat{\mathbf{S}}_{\text{filtered}}| \times d^a}$,
and see $\hat{\mathbf{S}}_{\text{filtered}}$ as paragraph *P:"⋯NtSetValueKey Critical."* is input to get *token-level embeddings* by Ω^{token}_e:

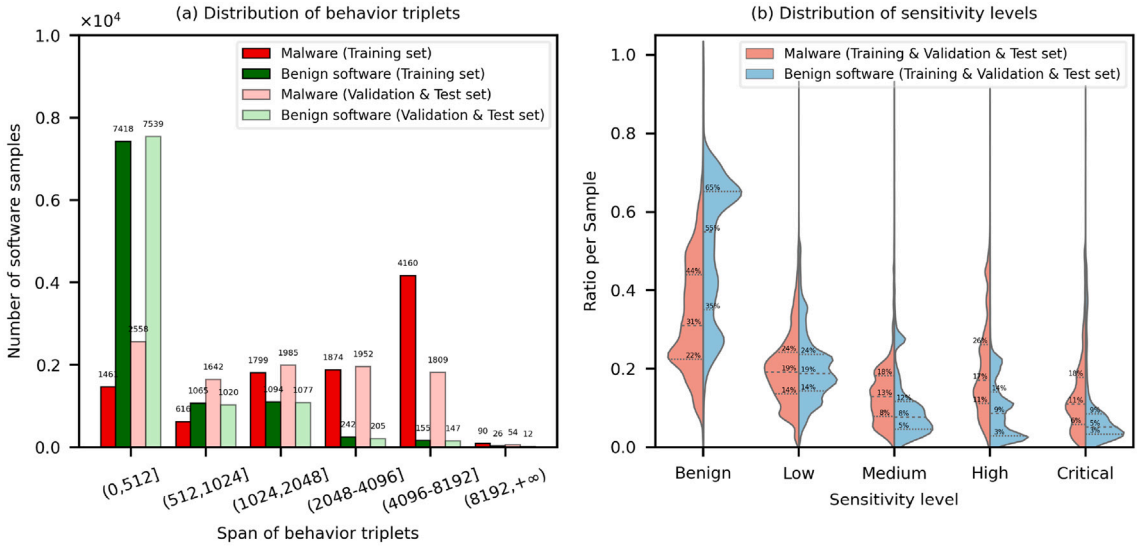


Fig. 2. Distribution of behavior triplets and sensitivity levels for dataset.

$$\boldsymbol{H} = \Omega_e^{token}(\text{“}\cdots / \underbrace{Nt,Set,Value,Key,Critical,.}_{j\text{-th token index span}[l_j^{st}, l_j^{ed}]} / \text{”}) \in \mathbb{R}^{L \times d^r}$$

$$= \begin{bmatrix} & 0.02 & 0.07 & 0.01 & 0.05 & -0.06 & 0.01 \\ \cdots & -0.05 & 0.02 & 0.03 & 0.04 & 0.07 & 0.03 \\ \cdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ & 0.07 & 0.03 & 0.05 & 0.10 & -0.04 & -0.02 \end{bmatrix},$$

then get each e_k^r of $\hat{s}_k \in \hat{\mathbf{S}}_{\text{filtered}}$, e.g.,

$$e_j^r = \frac{1}{l_j^{ed} - l_j^{st} + 1} \sum_{\alpha = l_j^{st}}^{l_j^{ed}} \boldsymbol{H}_\alpha$$

$$= \frac{1}{6}\left(\begin{bmatrix} 0.02 \\ -0.05 \\ \vdots \\ 0.07 \end{bmatrix} + \cdots + \begin{bmatrix} 0.01 \\ 0.03 \\ \vdots \\ -0.02 \end{bmatrix} \right) = \begin{bmatrix} 0.0167 \\ 0.0233 \\ \vdots \\ 0.0317 \end{bmatrix} \in \mathbb{R}^{d^r},$$

so $\boldsymbol{E}^{itr} = [\cdots, e_j^r] = [\cdots, [0.0167, \cdots, 0.0317]] \in \mathbb{R}^{|\hat{\mathbf{S}}_{\text{filtered}}| \times d^r}$.

This step extracts fine-grained intent embeddings and context embeddings from local and sequential dimensions.

Step 5: concatenate $\boldsymbol{E}^{ita}$ and $\boldsymbol{E}^{itr}$ to $\boldsymbol{E}^{+}$:

$$\boldsymbol{E}^{+} = [\cdots, e_j^r \| e_j^a] = [\cdots, [0.0167, \cdots, -0.0615]] \in \mathbb{R}^{m \times (d^r + d^a)},$$

where it fuses dual-view embeddings into a unified representation for precise and robust downstream detection.

4.2. Applicable scenario and deployment mode

We emphasize that *SaBER-LLMs* is **not** designed as an inline, hard real-time blocking component. Instead, it primarily serves as a semantic enrichment and representation stage deployed in an **analysis backend**, such as sandbox-based dynamic malware analysis or quasi-offline detection. In practical dynamic analysis, malware samples are typically executed and monitored in a sandbox for a minutes-level time window to capture sufficiently rich behavioral traces, where the monitoring duration is configurable and depends on the analysis goal and sandbox policy [23]. Such a setting is inherently different from inline endpoint blocking, and the overall analysis timescale is dominated by trace collection rather than the downstream classifier inference.

Where does our preprocessing fit? The LLM-based processing can be integrated in two practical ways: (i) *Pipeline/ streaming mode:* perform semantic translation, sensitivity rating, and embedding extraction during the sandbox monitoring window as API events are produced; (ii) *Post-trace mode:* execute preprocessing immediately after the trace is collected, as part of the backend enrichment workflow.

Therefore, the backend preprocessing stage may incur a seconds-level overhead, which should be interpreted relative to the minutes-level sandbox monitoring window and is typically more acceptable in backend analysis/detection scenarios. We provide a quantitative breakdown in Section 5.

5. Experiments

5.1. DataSet, setup and evaluation metrics

5.1.1. DataSet

Our dataset, accessible via GitHub,[2] comprises 20k benign samples drawn from *DataSet-Stage1* and 20k malicious samples including 10k from *Stage1* and an additional 10k selected randomly from *Stage2*. We divide it into a training set (20k), validation set (4k), and test set (16k), each with the same number of benign and malicious samples.

- *Distribution of behavior triplets.* Statistical data from the training and validation datasets indicate 2,537,350 triplets, with 2,023,565 from malware, 445,085 from benign software, and 34,350 shared in both, and the test dataset has 1,456,201 de-duplicated triplets, showing a vast state space and significant distribution differences. Fig. 2(a) shows the behavior triplets of malware, compared to benign software, are typically more extended, indicating a larger state space.
- *Distribution of sensitivity levels.* Through Step 1&2, we quantified each triplet's sensitivity in Fig. 2(b), showing that both benign and malicious software involve behaviors at all sensitivity levels, but malware presents a noticeably higher ratio of high-risk behaviors (e.g., High and Critical), highlighting behavior semantic distinctions.

5.1.2. Experimental setup

Our experiments are implemented on 13th Gen Intel(R) Core(TM) i7-13700KF CPU with two NVIDIA GeForce RTX 2080Ti GPUs and 32GB

[2] https://github.com/kericwy1337, including *Stage1* and *Stage2*.

Table 5
Hyperparameter Settings.

Module	Name	Value
Prompt-based semantic translation	Ω_p	qwq:32b[a]
	l_{max}	50
Prompt-based sensitivity rating	Ω_s	qwq:32b[a]
Two-stage state compression	τ	Medium
Dual-view embeddings	Ω_e	qwen3-embedding:8b[b]
	d^a	64
	Ω_e^{token}	qwen3-embedding:0.6b[b]
	d^r	64
Baseline classifiers	Epoch size[c]	64
	Batch size	16

[a] qwq:32b is a general-purpose instruction-following LLM with 32B parameters, which can rival DeepSeek-R1 with 671B parameters, chosen under moderate computational cost.

[b] qwen3-embedding:8b/0.6b are embedding variants of Qwen3 with 8B and 0.6B parameters, selected to balance embedding quality and efficiency in sentence and token representation.

[c] We adopt an early stopping mechanism, i.e., training will halt if validation accuracy fails to improve for 10 epochs.

RAM, as well as a 2.10GHz Intel(R) Xeon(R) CPU with 2 TITAN XP GPUs and 32GB RAM. The main hyperparameter settings are shown in Table 5. For baseline networks, Adam [22] is used for training, and the cross entropy loss function is as follows:

$$\mathcal{L}_{model} = -\frac{1}{|S|}\sum_{i=1}^{|S|}(y_i \log(p_i) + (1-y_i)\log(1-p_i))$$

where y_i is the ground-truth and p_i is its predicted probability. In addition, *SaBER-LLMs* supports parallelism, so we opened up multiple threads (128) for each step's execution.

LLM decoding settings. All LLM interactions in *SaBER-LLMs* adopt deterministic decoding to ensure stability and reproducibility. We set the temperature to 0, and disable sampling, so that the same input produces identical outputs, cap the maximum output length (l_{max}) to avoid overly verbose responses, and use structured prompts to constrain the output format, which eliminates randomness in semantic translation and sensitivity rating, making the downstream filtering and embedding deterministic and repeatable.

LLM fallback correction. To address occasional incomplete or malformed outputs, we apply lightweight validation rules to ensure format consistency and required fields. If an output violates the expected structure (e.g., missing sensitivity labels or malformed intent descriptions), we automatically re-query the LLM with the same prompt, up to a maximum of three attempts. If the issue persists, a conservative fallback strategy is adopted by assigning a higher sensitivity level (e.g., *Critical*), preventing underestimation of security risks and ensuring robustness, which is designed to handle technical instability, rather than semantic certainty.

5.1.3. Evaluation metrics

We evaluate *SaBER-LLMs* based on both detection effectiveness and computational efficiency. For effectiveness, we report Accuracy, True Positive Rate (TPR), True Negative Rate (TNR), Precision, Recall, F1-score, and Area Under Curve (AUC). For efficiency, we decompose runtime metrics into three components: (i) *processing time* T_{proc} for LLM-based semantic enrichment and representation construction (executed in an analysis backend), (ii) *training time* T_{train} for the downstream classifier, and (iii) *detection (inference) time* T_{infer} for prediction after the representation is obtained. This separation prevents confusion between preprocessing overhead and inference latency, supports fair comparisons under different processing assumptions, and helps identify where costs and speedups arise.

5.2. Model evaluation

To verify the effectiveness of *SaBER-LLMs*, we conduct three experimental groups with progressively stronger representations, all evaluated under identical downstream classifiers. Specifically, (*i*) *Word2Vec-N / NPR* uses Word2Vec to embed API call sequences over **n**ame only or **p**arameters and **r**eturn value jointly, showing the effect of incorporating fine-grained information; (*ii*) *BERT-N / NPR* replaces Word2Vec with BERT, enabling a fair comparison between static and contextualized encoders under comparable input information; and (*iii*) *SaBER-LLMs* is trained with the security-aware behavior embedding $\boldsymbol{E}^+$. For all experimental groups, we repeat training five times with different random seeds and report the averaged results to ensure statistical stability.

As illustrated in Table 6, we compare (*iii*) *SaBER-LLMs* with (*i*) static embedding *Word2Vec-NPR* and (*ii*) dynamic embedding *BERT-NPR* under comparable input configurations. Across both downstream classifiers, (*iii*) consistently achieves the best overall performance. When compared with the stronger *BERT-NPR*, *SaBER-LLMs* improves accuracy from 91.72% to 97.71% (+5.99%) for Text-CNN and from 89.58% to 96.43% (+6.85%) for BiGRU-Attention. AUC is also consistently higher, i.e., 99.59% vs. 98.19% (+1.40%), and 98.94% vs. 95.78% (+3.16%). Meanwhile, T_{infer} is significantly reduced, i.e., 3.74 ms vs. 31.22 ms for Text-CNN (−27.48 ms), and 10.88 ms vs. 42.11 ms for BiGRU-Attention (−31.23 ms), demonstrating that the security-aware representation $\boldsymbol{E}^+$ provides substantial and consistent gains.

Further insights are obtained within each group by comparing the *-N* and *-NPR* variants. For *Word2Vec*, introducing parameters and return values does not improve performance; instead, it leads to unstable behavior characterized by high precision but severely degraded recall (around 42%), resulting in a lower F1-score and increased T_{infer}. Similarly, for *BERT*, extending the input from *-N* to *-NPR* fails to yield improvements and even causes noticeable drops in accuracy and efficiency. These observations suggest that naively appending parameters and return values as raw lexical tokens may introduce noise and sparsity, enlarging the state space without effectively capturing their contextual security semantics. As a result, simply enriching the input with fine-grained fields does not guarantee better generalization and may instead aggravate overfitting or convergence instability.

In contrast, *SaBER-LLMs* does not treat parameters and return values as the plain strings, but interprets them through LLM-guided semantic translation and sensitivity-aware abstraction. By transforming the low-level API call triplets into security-relevant behavioral intent and filtering redundant or low-risk signals before downstream modeling, *SaBER-LLMs* effectively compresses the semantic state space while preserving discriminative information. The principled semantic abstraction explains why $\boldsymbol{E}^+$ outperforms both the dynamic and static embeddings, showing that the key advantage lies not merely in stronger encoders or richer input tokens, but in structured security-aware representation learning.

5.3. Model comparison

To demonstrate the superiority of *SaBER-LLMs*, we compare our two baselines with six existing SOTA methods (considering parameters and return values).

Fang et al. [12] use hashing to encode API names, modules, and return values, and then select key features for XGBoost.

Zhang et al. [50] hash API names, categories, parameters, and return values into features, then feed them to CNN-BiLSTM.

Chen et al. [8] mark API calls with parameter sensitivity using empirical rules,[3] statistics, and GSDMM, and concatenate embeddings for Text-CNN and BiLSTM-Attention.

[3] Due to the inaccessibility of the authors' manually crafted rules (over one hundred in total), we substitute them with Yara rules [42] in our reproduction.

Table 6
Evaluation of two baseline networks under three experimental groups for test results ($\tau = Medium$).

Item	Setting	Baseline Networks	Detection Effectiveness (%)					T_{infer} (ms)	Change (Compared to (*iii*))		
			Acc.	Precision	Recall	F1-score	AUC		Acc.	AUC	T_{infer}
(*i*)	*Word2Vec -N*	Text-CNN	66.17	66.89	64.04	65.43	66.89	7.31	**−31.54**	**−32.70**	**+3.57**
		BiGRU-A	80.69	75.23	91.51	82.58	81.31	12.53	**−15.74**	**−17.63**	**+1.65**
	Word2Vec -NPR	Text-CNN	71.31	100.00	42.61	59.76	98.27	25.06	**−26.40**	**−1.32**	**+21.32**
		BiGRU-A	70.70	99.88	41.45	58.59	96.64	37.66	**−25.73**	**−2.30**	**+26.78**
(*ii*)	*BERT -N*	Text-CNN	93.17	96.36	89.72	92.93	98.06	8.68	**−4.54**	**−1.53**	**+4.94**
		BiGRU-A	91.71	97.88	85.28	91.14	96.91	14.86	**−4.72**	**−2.03**	**+3.98**
	BERT -NPR	Text-CNN	91.72	97.33	85.79	91.20	98.19	31.22	**−5.99**	**−1.40**	**+27.48**
		BiGRU-A	89.58	97.91	80.88	88.58	95.78	42.11	**−6.85**	**−3.16**	**+31.23**
(*iii*)	*SaBER -LLMs*	Text-CNN	**97.71**	98.87	96.51	97.68	**99.59**	3.74	/		
		BiGRU-A	**96.43**	98.44	94.35	96.35	**98.94**	10.88	/		

Table 7
Performance comparison with six existing state-of-the-art approaches ($\tau = Medium$).

Approach	Model	Test Results (%)				T_{proc}	T_{train}	T_{infer}
		Accuracy	TPR	TNR	F1-score	(ms/sample)		
Fang et al. [12]	XGBoost	88.41	83.31	93.51	87.79	112.76	64.60	49.13
Zhang et al. [50]	CNN-BiLSTM	89.38	87.69	91.06	89.19	292.22	28.55	18.28
Chen et al. [8]	Text-CNN	91.56	90.80	92.33	91.50	189.08	18.27	16.14
	BiLSTM-Attantion	90.69	89.92	91.45	90.62		27.50	21.65
Jing et al. [18]	BERT + SVM	93.88	97.19	91.02	93.66	509.82	11.54	9.61
	BERT + Text-CNN	94.18	95.03	93.37	94.13		3.65	2.84
	BERT + BiGRU-Attantion	96.33	97.16	95.54	96.30		12.04	6.71
Cui et al. [10,11]	Doc2Vec + KNN	94.67	93.70	95.62	94.58	339.50	60.97	12.18
	BERT + Text-CNN	96.25	94.88	97.61	96.18	795.84	107.72	29.65
Shahnawaz et al. [37]	Image + CNN	89.96	86.05	93.88	89.55	191.75	27.00	0.65
SaBER-LLMs	**Text-CNN**	**97.71**	**96.51**	**98.90**	**97.68**	12,020[a]	28.75	3.74
	BiGRU–Attantion	**96.43**	**94.35**	**98.5**	**96.35**		48.75	10.88

[a] T_{proc} (12.02s) that can be integrated in a sandbox-based analysis backend is more acceptable relative to the overall analysis timescale (often 2∼10 mins), while still providing substantial gains in detection accuracy, interpretability, and robustness. Concretely, it consists of *step1 semantic translation* (3.93s) + *step2 sensitivity rating* (2.52s) + *step3 state compression* (33.75ms) + *step4 intra-call & inter-call embedding* (3.09s + 2.40s) + *step5 embedding combination* (45.77ms).

Jing et al. [18] infer sensitive intents and crucial snippets through brain-inspired functions, using BERT to obtain embeddings.

Cui et al. [10,11] model API behaviors by constructing temporal process graphs and API property graphs, generating behavior paths via random walks, and learning embeddings using Doc2Vec/BERT, which are then used for KNN/Text-CNN.

Shahnawaz et al. [37] convert runtime API call parameters and return values into grayscale images and apply CNN to classify malware.

All experiments are conducted on our dataset, and are set up according to the corresponding paper. We also record the average processing, training, and detection time per sample.

From Table 7, *SaBER-LLMs* consistently outperforms all six existing state-of-the-art approaches in terms of detection accuracy and F1-score. Notably, Text-CNN and BiGRU-Attention achieve 97.71% and 96.43% accuracy respectively, significantly surpassing the previous best results of 88.41% [12], 89.38% [50], 91.56% [8], 96.33% [18], 96.25% [11], and 89.96% [37], which verifies the effectiveness of *SaBER-LLMs* in capturing deeper security-aware intent, even in fine-grained behavioral variance. This is attributed to the LLM-guided semantic translation and sensitivity rating, which abstract low-level API or parameter variations into higher-level security intent and prioritize security-relevant evidence.

For time consumption, we report a decomposed cost in terms of T_{proc}, T_{train}, and T_{infer}. Since *SaBER-LLMs* performs processing on the entire sequence to ensure a more comprehensive analysis, unlike [8] which uses only top 1000 calls, it incurs a higher overhead in T_{proc}, i.e., 12.02s per sample. This is due to LLMs interactions for semantic translation (3.93s), sensitivity rating (2.52s), and intra/inter-call embeddings (3.09s/2.40s). Importantly, T_{proc} corresponds to backend preprocessing and can be integrated into sandbox-based dynamic analysis (e.g., pipeline/streaming during trace collection or immediately after it). Given that sandbox monitoring typically operates at a minutes-level window (often 2∼10 mins), a seconds-level T_{proc} (12.02s in our setting) is more practically acceptable in such backend workflows. However, the T_{train} and T_{infer} remain efficient and comparable to existing methods, namely, 28.75ms and 3.74ms of Text-CNN, 48.75ms and 10.88ms of BiGRU-Attention, compared to 12.04ms and 6.71ms [18]. Overall, the additional backend preprocessing cost is outweighed by the substantial performance gains and improved generalizability.

End-to-end throughput. Given T_{proc} (12.02s/sample), a single-threaded execution yields about 300 samples/hour. In practice, the LLM-based preprocessing is clearly parallel and can be implemented by a multi-threaded, asynchronous backend. In our implementation, we set 128 parallel threads, allowing semantic translation, sensitivity rating, and embedding extraction to be executed concurrently across samples. Under such a realistic backend configuration, the end-to-end throughput can exceed several thousand samples (e.g., ∼4k) per hour, which is well aligned with sandbox-based dynamic analysis that operates at a minute-level analysis window.

Computational and monetary cost. The overhead mainly consists of two parts: (i) semantic translation and sensitivity assessment using Ω_p&Ω_s (remote-access qwq:32b via API key), and (ii) intra-/inter-call embedding generation using Ω_e&Ω_e^{token} (locally deployed qwen3-embedding:8b&0.6b). For (i) on 40k samples, the total # of input tokens consumed by qwq:32b is 1,742,782,340, and the # of output tokens is 652,502,651. Referring to the official pricing of qwq:32b[4] (2 RMB & 6

[4] https://help.aliyun.com/zh/model-studio/model-pricing

Table 8
Impact of l_{max} on LLM-generated security descriptions and sensitivity assessment under fixed Ω_p & Ω_s and decoding settings.

API call	Ω_p & Ω_s	l_{max}	Security Description[a]	Sensitivity[a]
① `Get Module File Name`	qwq:32b	25	Retrieves the path of the specified module, here "UuYUIQAM.exe".	*Low*
		50	The operation retrieves the full path of the specified module "UuYUIQAM.exe" when the first parameter is NULL, likely indicating the process itself or another module within the process.	*Low*
		100	The operation likely represents a call to retrieve the full path of the current process's executable file, which is named "UuYUIQAM.exe" and located in an uncommon directory "C:\Documents and Settings\All Users\pAgIwsIY". This be-havior may indicate potential activity due to the unusual file path and name.	*Low*
② `Copy File ExW`	qwq:32b	25	Software installation copying an executable to system32, possibly malicious.	*Medium*
		50	An executable file is being copied from a non-standard user-accessible location to the system32 directory, potentially allowing it to run with higher privileges or ensuring its persistence on the system. This activity could indicate software installation or malicious behavior and requires further investigation.	*High*
		100	An executable file is being copied from a subdirectory within the All Users profile to the system32 directory, which contains critical system files. This action requires administrative privileges and is potentially risky, as it could allow un-authorized modifications to system components. The use of obfuscated folder and file names increases suspicion of malicious intent, such as installing malware or persistent access. This activity should be investigated to confirm its legitimacy.	*High*
③ `Set File Hidden Or Read Only`	qwq:32b	25	The operation likely involves modifying the file attributes of "iwkg.exe" to make it hidden or read-only, possibly for security or system integrity purposes.	*Medium*
		50	The operation likely represents an attempt to hide and protect a file named "iwkg.exe" located in the system32 directory by setting it as hidden and read- only. This could be a sign of malware trying to conceal its presence.	*High*
		100	The operation involves setting the file "C:\WINDOWS\system32 \iwkg.exe" with attributes that make it hidden, read-only, and system, likely using the SetFileAttributes API. This behavior is suspicious as it allows the file to be hidden and prevents easy modification or deletion, which is a common tactic used by malware to persist on a system. The success of this operation indicates that the initiating process had sufficient privileges to modify files in the sensitive system32 directory.	*High*

[a] Reduce the output randomness by setting the temperature to 0 and disabling sampling.

RMB per million input tokens and output tokens), the cost is approximately 3485 RMB & 3915 RMB for input and output tokens, resulting in a total of 7400 RMB, which corresponds to an average token-level cost of about 0.1859 RMB/sample under API key-based pricing. For (ii) on local deployment, the cost mainly lies in electricity consumption. The embedding stage needs $(3.09\text{s}+2.40\text{s})\times 40k \approx 219{,}600\text{s}$ (61 hours) in total under sequential estimation. Our server power consumption is 0.75 kW, so the total energy usage is about 45.75 kWh. Given the electricity price of 0.8 RMB per kWh, the corresponding electricity cost is about 36.6 RMB. Overall, the estimated total cost for processing 40k samples is 7,436.6 RMB, leading to an average cost of roughly **0.1860 RMB/sample** under our deployment, where the estimation is a conservative upper bound. In practice, several strategies discussed in Section 7 can further reduce amortized cost to provide flexible trade-offs between semantic richness and computational budget.

5.4. Ablation study

To explore the effects and gains of various configurations, ablation studies are conducted as the basis for the final structure. Note that while the choice of prompts, LLMs, and Embedding-LLMs may influence intermediate outputs, a comprehensive exploration of all possible prompt designs and LLM combinations would require substantial additional effort and is beyond the scope of this paper. Our goal here is to provide representative and controlled evidence demonstrating that the proposed framework exhibits reasonable stability under practical variations. We first investigate the stability of the prompt engineering to analyze how variations in l_{max} and Ω_p&Ω_s influence the generated security descriptions and sensitivity labels, which are important to the reproducibility of *SaBER-LLMs*. Then, we examine the influence of τ on sequence compression, convergence behavior, and detection accuracy, and finally assess the robustness of *SaBER-LLMs* across different test distribution spans.

5.4.1. Impact of l_{max}

To study the impact of l_{max} on semantic translation and sensitivity assessment, we vary l_{max} in the prompt template. Specifically, we consider $l_{max} \in \{25, 50, 100\}$ while keeping other variables fixed, including Ω_p&Ω_s and decoding settings. We report the generated security descriptions and sensitivity labels for three API calls (①∼③) from the snippet in Section 2.1. This setup allows us to directly observe how description granularity affects semantic interpretation and downstream sensitivity judgment.

From Table 8, we observe that when l_{max} is small (i.e., 25), the generated security descriptions tend to be overly concise and may omit important contextual cues, which can occasionally lead to less stable sensitivity assessments. As l_{max} increases to a moderate value (i.e., 50), the descriptions consistently capture key security-relevant semantics, and the sensitivity labels become stable across different API calls. Further increasing l_{max} to 100 yields more verbose descriptions with finer details, but does not significantly change the sensitivity. Instead, longer descriptions introduce additional computational overhead in subsequent embedding extraction without clear semantic benefits. Based on this trade-off between semantic completeness, stability, and efficiency, we select 50 as the default configuration in our experiments.

Table 9
Impact of Ω_p & Ω_s on semantic translation and sensitivity assessment with identical prompts and decoding configurations.

API call	l_{max}	Ω_p & Ω_s	Security Description[a]	Sensitivity[a]
① `Get Module File Name`	50	GPT5.2	The process queries the full filesystem path of its own executable, likely to deter-mine its installation location under All Users, often for persistence checks, self-replication, or environment awareness.	*Low*
		qwq:32b	The operation retrieves the full path of the specified module "UuYUIQAM.exe" when the first parameter is NULL, likely indicating the process itself or another module within the process.	*Low*
		DeepSeek-V3:671B	A process retrieving its own executable path to potentially verify its location or for persistence purposes within a common user directory.	*Low*
② `Copy File ExW`	50	GPT5.2	The process copies an executable from a user-accessible directory into the system-32 folder under a new name, likely to establish persistence or masquerade as a legi-timate system file, indicating potentially malicious installation or privilege abuse.	*High*
		qwq:32b	An executable file is being copied from a non-standard user-accessible location to the system32 directory, potentially allowing it to run with higher privileges or ensuring its persistence on the system. This activity could indicate software installation or malicious behavior and requires further investigation.	*High*
		DeepSeek -V3:671B	A suspicious file copy operation from a user directory to the Windows system32 folder, likely part of malware establishing persistence.	*High*
③ `Set File Hidden Or Read Only`	50	GPT5.2	The process attempts to change file attributes on a system32 executable, likely to hide or protect it from modification. The return value suggests the attribute change operation completed as intended.	*Medium*
		qwq:32b	The operation likely represents an attempt to hide and protect a file named "iwkg.exe" located in the system32 directory by setting it as hidden and read-only. This could be a sign of malware trying to conceal its presence.	*High*
		DeepSeek-V3:671B	A process is likely attempting to hide and mark the system file "iwkg.exe" as read-only to prevent its detection, modification, or deletion.	*High*

[a] Reduce the output randomness by setting the temperature to 0 and disabling sampling.

5.4.2. Impact of Ω_p & Ω_s

To examine the influence of different LLMs on semantic translation and sensitivity assessment, we evaluate three representative LLMs as Ω_p and Ω_s: GPT5.2, qwq:32b, and DeepSeek-V3:671B. We fix $l_{max} = 50$ and keep all other configurations unchanged, including prompt templates and decoding settings. By comparing the security descriptions and sensitivity labels, we aim to assess the robustness of the proposed framework with respect to the choice of LLMs, rather than optimizing performance for a specific model.

As shown in Table 9, the security descriptions generated by different LLMs are largely consistent in terms of semantic intent, even though their phrasing and level of detail vary. For ①∼②, three LLMs produce identical sensitivity labels, indicating stable risk assessment across LLM choices. A minor discrepancy is observed for ③, where GPT5.2 assigns a slightly lower sensitivity level than qwq:32b and DeepSeek-V3:671B. Considering the overall consistency, together with computational efficiency and deployment cost, we choose qwq:32b as the default LLM for semantic translation and sensitivity assessment in *SaBER-LLMs*. These results suggest that the framework is not overly sensitive to the specific choice of LLM and exhibits reasonable robustness under practical model variations.

5.4.3. Influence of τ

The high quality of *filtering strategy* is the prerequisite for the superior performance of *SaBER-LLMs*, in which τ is the most key factor affecting the compression effect, so we set $\tau \in Y^{sl}, \Delta_\tau = 1$ to observe its influence in Fig. 3, mainly from the following four perspectives:

- *Behavior triplet compression and sample distortion.* From Fig. 3(a), the average # of triplets per sample decreases dramatically as τ increases, effectively reducing the semantic state space and highlighting more sensitive behaviors. However, an overly strict threshold (e.g., *High* or *Critical*) may cause sample distortion, especially for benign samples lacking high-sensitivity behaviors, leading to their complete removal. Thus, selecting an appropriate τ is crucial to balance effective space reduction without incurring excessive sample loss.
- *Processing efficiency.* From Fig. 3(b), the overall processing time decreases with increasing τ. While Step 1 (semantic translation) and Step 2 (sensitivity rating) remain constant across thresholds, the majority of time savings occur in Step 4 (intra/inter-call embedding generation), which scales with sequence length. This validates the efficiency gain from filtering irrelevant or low-impact behaviors early in *SaBER-LLMs*.
- *Training dynamics.* From Fig. 3(c), the optimal training epoch exhibits a U-shaped trend relative to τ, reaching its minimum when τ is set to *Medium*. This reflects two underlying effects: ① insufficient compression (low τ) leads to overly long, noisy sequences that hinder convergence; ② overly aggressive filtering (high τ) removes informative cues, increasing classification difficulty. In contrast, training time consistently declines with stricter thresholds due to shorter input sequences.
- *Detection performance.* From Fig. 3(d), the test accuracy of both baselines initially improves with increasing τ, peaking at *Medium* (97.71% and 96.43%), then slightly declines, which suggests that a moderate threshold strikes a favorable trade-off between retaining discriminative features and ensuring convergence. Meanwhile, detection time follows a downward trend, aligned with the reduced input length, further supporting the utility of state compression in accuracy and efficiency.

Consequently, we use τ=*Medium* as the optimal setting, as it achieves the highest detection accuracy on both baselines while simultaneously maintaining stable convergence and efficient processing time, thus offering a well-balanced trade-off between semantic retention and space reduction.

5.4.4. Impact of test distribution

Referring to the distribution of triplets, the test accuracy of *SaBER-LLMs* is assessed across diverse distributions by dividing the dataset.

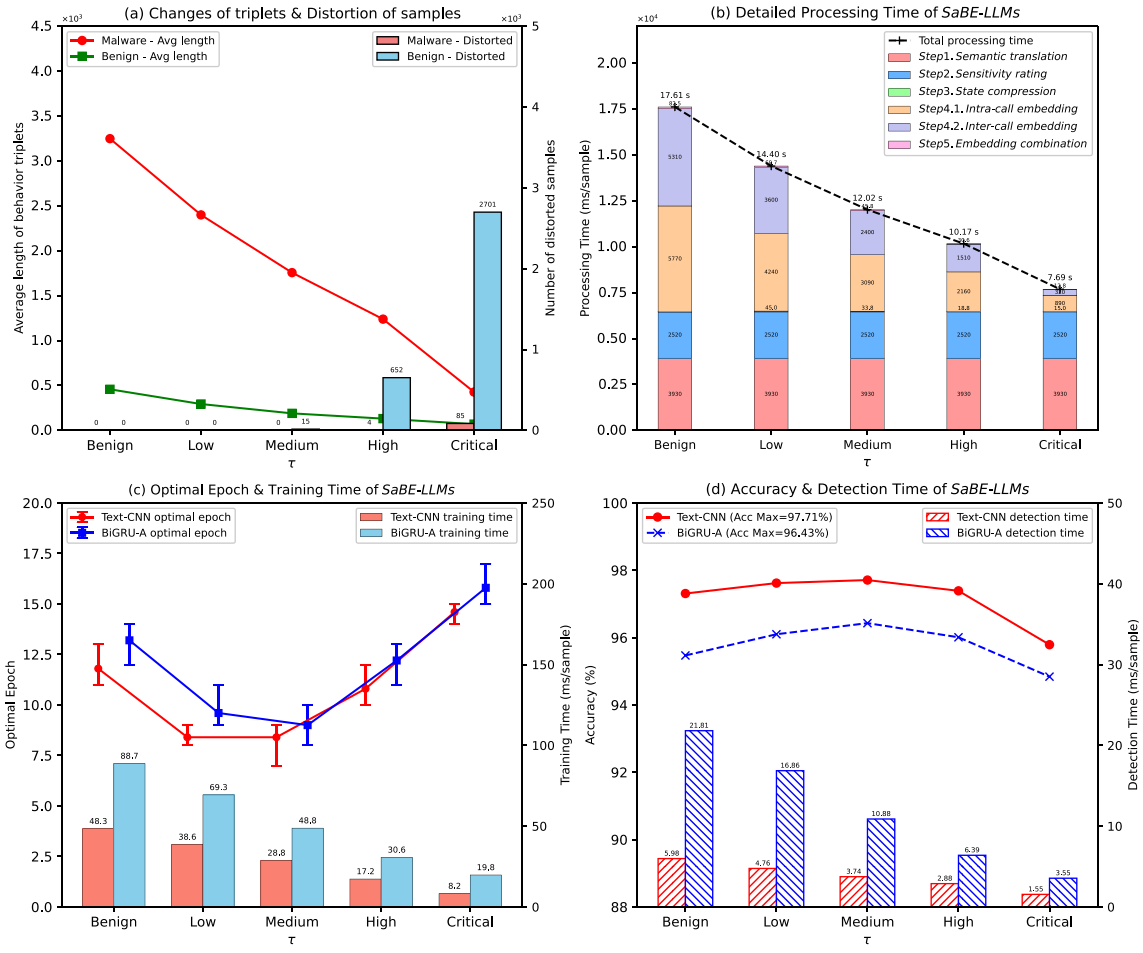


Fig. 3. Influence of the sensitivity threshold $\tau \in Y^{sl}, \Delta_\tau = 1$ on the processing, training, and test processes for *SaBER-LLMs*.

Table 10
Impact of test dataset distribution ($\tau = Medium$).

Spans[a]	<512	~1024	~2048	~4096	~8192	>8192
Type	**Proportion of samples with different triplet lengths (%)**					
Malicious	8.12	3.86	9.16	9.78	18.81	0.29
Benign	37.7	5.18	5.36	0.98	0.73	0.06
Baselines	**Test accuracy with different behavior triplet lengths (%)**					
Text-CNN	97.91	94.54	95.87	97.97	99.65	98.21
BiGRU-A	91.87	89.90	94.62	96.63	99.04	89.29

[a] ~1024 means # of triplets ∈ (512, 1024], and the rest are similar.

In the upper part of Table 10, there is a significant disparity in behavior triplet distributions between malicious and benign samples. Benign samples are mainly concentrated in shorter spans (37.7% ∈ (0,512]), while malware exhibits a broader and heavier-tailed distribution (19.1%∈ (4092, +∞]). This observation highlights the inherent complexity and verbosity of malicious behaviors, suggesting that truncation-based techniques may fail to retain critical patterns.

In the lower part of Table 10, *SaBER-LLMs* demonstrates consistent effectiveness on various triplet spans, yet notable variations still exist. Text-CNN and BiGRU-Attention maintain high accuracy overall, but are slightly degraded in mid-length span (512,1024], where semantic ambiguity and class imbalance may increase. Conversely, accuracy improves significantly in longer span (4096,8192], especially reaching 99.65% and 99.04%, suggesting that richer behavioral contexts enhance the discriminative power of *SaBER-LLMs*. This trend further confirms its robustness to longer sequence variability, while also emphasizing the challenge posed by medium-length sequences that may lack sufficient distinguishing signals.

5.5. Gains of SaBER-LLMs

Factually, *SaBER-LLMs* is composed of three conceptually distinct and independently adjustable components: **(i)** *intra-call embedding*, which captures fine-grained semantics within individual API calls; **(ii)** *inter-call embedding*, which models cross-call behavioral dependencies along the execution trace; and **(iii)** *compression strategy*, which performs sensitivity filtering to retain security-relevant behaviors while suppressing redundant, irrelevant or low-risk signals. To explicitly isolate the contribution of each component and their combinations, we design a controlled ablation study with seven experimental settings:

①*Without (i)&(ii)&(iii)*. It removes all components and is the same as *BERT-NPR* in Section 5.2. The input sequence (including API call name, parameters, and return value) is embedded by pre-trained BERT, without the semantic translation, sensitivity filtering, or dual-view embedding, serving as the baseline for assessing the gains of each component.

②*With (i) only*. It uses *intra-call embedding* alone by $\boldsymbol{E}^{ita}$, without inter-call modeling or compression, which isolates the semantic translation effect of the individual API call, answering whether enriching it with fine-grained semantics is beneficial even without considering inter-call dependencies.

③*With (i)&(iii)*. It combines *intra-call embedding* with *compression strategy* by applying sensitivity filtering before extracting $\boldsymbol{E}^{ita}$ from $\hat{\mathbf{S}}_{\text{filtered}}$, which isolates the compression effect on intra-call semantics

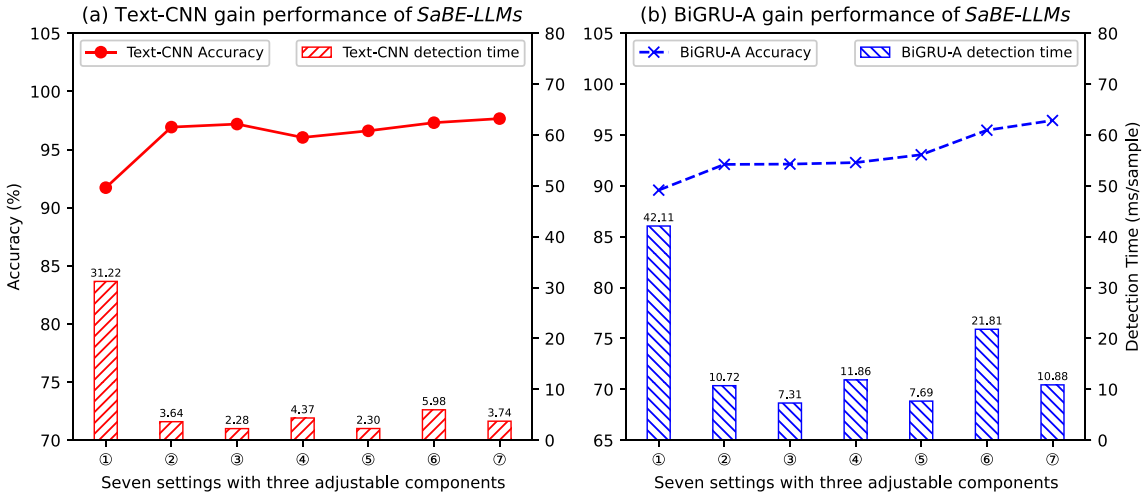


Fig. 4. Gain performance comparison of seven settings ($\tau = Medium$).

by comparing with ②, showing whether removing low-risk behaviors improves the discriminability of fine-grained semantic representations.

④*With (ii) only*. It uses *inter-call embedding* alone by $\boldsymbol{E}^{itr}$, without intra-call semantics or compression, which evaluates whether modeling behavior dependencies across calls improves detection performance even without detailed intra-call semantic abstraction.

⑤*With (ii)&(iii)*. It combines *inter-call embedding* with *compression strategy* by extracting $\boldsymbol{E}^{itr}$ from $\hat{\mathbf{S}}_{\text{filtered}}$, which isolates the compression contribution when applied to inter-call behavior modeling by comparing with ④, highlighting its role in suppressing noisy dependencies and emphasizing security-relevant transitions.

⑥*With (i)&(ii)*. It enables both *intra-call* and *inter-call embeddings* without *compression strategy* by $\boldsymbol{E}^{+}$ from $\mathbf{S}$, which evaluates whether dual-view semantic modeling alone is sufficient to capture malicious intent and intra-&inter-call semantics provide complementary information.

⑦*With (i)&(ii)&(iii)*. It uses all components and corresponds to (*i*) in Section 5.2. Comparing it with ⑥ directly isolates the added value of compression on top of dual-view semantic modeling, and represents the final configuration.

In all settings, the downstream classifier architectures and training protocols are kept identical, and only the tested components are varied. Comparing them in pairs to isolate each component can help understand their unique impact and collective benefits. The ablation settings are organized to answer three key questions:

Q1. What is the gain of introducing semantic embeddings beyond raw BERT features?

Q2. How does sensitivity-aware compression affect representation quality and robustness?

Q3. Are intra-&inter-call embeddings complementary, and how much do they contribute individually versus jointly?

From Fig. 4, two baselines exhibit clear accuracy and efficiency improvements as different components of *SaBER-LLMs* are progressively introduced, even when ① already incorporates API call names, parameters, and return values via Word2Vec, indicating that the observed gains are not merely due to richer input features, but stem from the structured semantic modeling introduced by the proposed components.

A1. Compare ② and ④ to ①. Enabling *intra-call embedding* alone (②) yields a substantial accuracy improvement of 5.21% for Text-CNN and 2.53% for BiGRU-Attention, demonstrating the effectiveness of LLM-guided semantic translation in capturing fine-grained security intent within individual API calls. Similarly, using *inter-call embedding* alone (④) also leads to notable gains (4.32% and 2.71%), indicating that modeling inter-call behavioral dependencies provides a complementary semantic perspective. These results confirm that both intra- and inter-call semantic embeddings significantly outperform raw BERT representations, even when parameters and return values are included.

A2. Compare ③ vs. ② and ⑤ vs. ④. In both cases, sensitivity filtering consistently reduces T_{infer} (e.g., from 3.64ms to 2.28ms and from 4.37ms to 2.30ms for Text-CNN, and from 10.72ms to 7.31ms and from 11.86ms to 7.69ms for BiGRU-Attention), while maintaining or slightly improving detection accuracy. This demonstrates that compression effectively suppresses low-risk or redundant behaviors, leading to more compact and discriminative representations with improved robustness and efficiency.

A3. Compare ⑥ to ② and ④, as well as to ⑦. The dual-view embedding (⑥) consistently outperforms either intra- or inter-call embedding alone, confirming that the two views capture complementary aspects of malicious behavior. Furthermore, the full integration (⑦) achieves the best overall performance across both baselines (97.66% for Text-CNN and 96.43% for BiGRU-Attention), while retaining competitive inference efficiency. This verifies that intra-call semantics, inter-call dependencies, and sensitivity-aware compression each contribute distinct benefits, and their combination yields a well-balanced and superior detection framework.

5.6. Out-of-distribution and temporal drift

A potential threat is *dataset contamination*: the 2019 public dataset may overlap with LLM pre-training data and confound memorization versus reasoning. We therefore add an out-of-distribution (OOD) evaluation on newly collected, temporally shifted malware samples.

Recent OOD datasets. We collect malware first submitted in *2024* and *2025* from VirusShare,[5] run each sample in a sandbox for *2 minutes*, and extract API call traces (names/parameters/returns) as behavior sequences. After filtering low-activity samples (< 10 calls), we retain 150 samples per year. These traces are newly generated and environment-specific, helping ensure the traces are pretraining-unseen.

Fig. 5 visualizes their distribution of sensitivity levels. Overall, both years are dominated by *Benign* and *Low* behaviors, while *Medium/High/Critical* occupy relatively small fractions. Notably, the 2025 set shows a clear shift towards slightly higher-risk composition compared with 2024, indicating non-trivial temporal drift even across adjacent years.

[5] https://virusshare.com

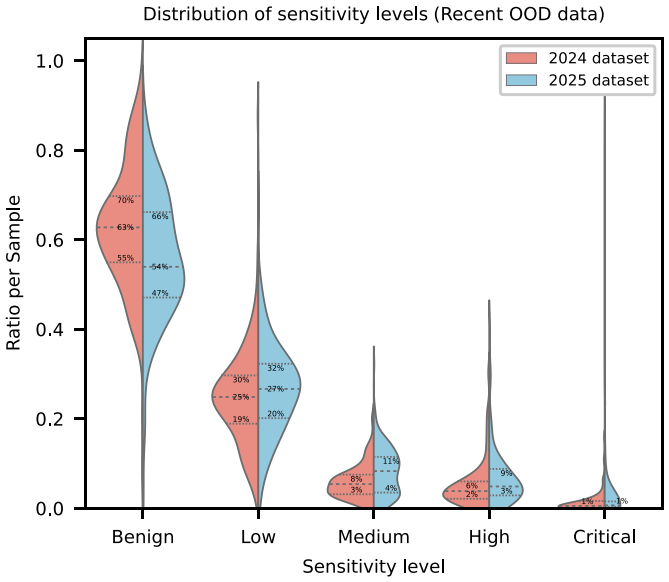


Fig. 5. Distribution of sensitivity levels for OOD dataset.

Table 11
Temporal drift of OOD dataset ($\tau = Medium$).

Baselines	OOD dataset (150 samples each year)			
	2024 set (%)		2025 set (%)	
	Accuracy	F1-score	Accuracy	F1-score
Text-CNN	99.33	99.67	99.33	99.67
BiGRU-A	100.00	100.00	99.33	99.67

Evaluation protocol under temporal drift. To test temporal robustness, we adopt a *train-on-2019, test-on-2024/2025* protocol, *without any retraining or fine-tuning*, which simulates a realistic deployment scenario where detectors must handle continuously evolving threats.

Table 11 reports the performance change across years. Despite the evident distribution shift, *SaBER-LLMs* consistently preserve strong superiority, achieving 99.33%~100% accuracy and 99.67%~100% F1-score, with negligible change from 2024 to 2025, which suggests that security-aware representations capture more *time-invariant* malicious semantics. The LLM-enabled semantic abstraction, together with sensitivity-guided compression, reduces reliance on surface-level API call patterns and makes the downstream classifier robust to temporal drift, thereby providing strong evidence of generalizability.

5.7. Adversarial attack

Adversarial attacks [25,35,48] in cybersecurity expose malware detectors to evasion via crafted API perturbations. To assess the robustness of *SaBER-LLMs*, we consider both *black-box* and *grey-box* attacks and adapt three types of perturbations, namely *API call injection*, *API call substitution*, and *parameter-level perturbation*. Before presenting specific attacks, we explicitly define the attacker assumptions.

Attacker assumptions. Table 12 summarizes the attacker assumptions under the black-box and grey-box settings. Both attackers are constrained to preserve malicious functionality and cannot arbitrarily rewrite execution traces, where the key difference lies in knowledge of *SaBER-LLMs*: the black-box attacker is unaware of the internal semantic abstraction or filtering mechanism and can only observe final outputs, whereas the grey-box attacker is aware of the existence of sensitivity levels and threshold-based filtering, but still has no access to prompts, embeddings, or model parameters. Neither attacker can directly manipulate the return values, as they are determined by the runtime system state.

Table 12
Attacker assumptions under black-box and grey-box settings.

Capability	Black-box	Grey-box
Must preserve functionality	✓	✓
Knowledge of prompt	×	×
Knowledge of sensitivity	×	✓
Knowledge of filtering (τ)	×	✓
Control over parameters	✓	✓
Control over return values	×	×

Adversarial metrics. We additionally introduce two metrics to evaluate the adversarial performance [18,19]:

① *Defense effectiveness* measures the proportion of malware in the test set that remains correctly classified as malicious after adversarial perturbation:

$$defense_effectiveness = \frac{|\{C(\mathbf{S}_i)=\text{mal.} \wedge C(\mathbf{S}_{adv,i})=\text{mal.}\}|}{|\{C(\mathbf{S}_i)=\text{mal.}\}|}$$

where C is a baseline classifier and mal. means malicious.

② *Attack overhead* quantifies the average percentage of API calls added or replaced to successfully misclassify:

$$attack_overhead = \frac{N(\text{ad/rp_APIs})}{\text{avg}(l)}$$

where $N(\text{ad/rp_APIs})$ is # number of added/replaced API calls, and l is the average length of sequences $\mathbf{S}$ misclassified from malicious to benign, i.e., $\mathbf{S} \in \{\mathbf{S} \mid C(\mathbf{S})=\text{mal.} \wedge C(\mathbf{S}_{adv})=\text{ben.}\}$.

5.7.1. API call injection attack

• *Black-box injection attack.* We adopt a generic, query-efficient no-op mimicry strategy [33,44], where the attacker injects *semantically irrelevant and minimally invasive* API calls to dilute malicious evidence, without any knowledge of the defender's risk-aware filtering or sensitivity categories. Hence, injected calls are randomly sampled from a broad candidate vocabulary, and only detector outputs are used to guide whether further perturbations are needed.

• *Grey-box injection attack.* We constrain injected API calls to a curated set that is likely to survive the threshold-based filtering under τ. Specifically, the attacker assumes knowledge of the existence of sensitivity levels and τ, and therefore avoids injecting API calls whose assessed sensitivity would be immediately filtered out. Instead, the attacker selects *irrelevant benign-trace API calls* whose sensitivity label aligns with or is higher than τ, so they are not trivially removed. This increases the chance that injected calls remain in the sequence and influence the final representation, while still requiring no-op and minimally invasive execution to preserve malware functionality.

Algorithm 2 unifies both settings via an attack-mode flag, where the only change is how the injection vocabulary V_{adv} is constructed. For black-box ($mode = black$), injected calls are sampled from V_{base}; for grey-box ($mode = grey$), injected calls are sampled from a risk-aware subset that is expected to survive the filtering threshold τ, thereby modeling a stronger attacker with partial knowledge of the defense mechanism. Here, $\textsc{Keep}(\cdot;\tau)$ indicates whether an API call is expected to be retained by the filtering/compression under threshold τ in the grey-box setting.

We exclude API calls that cannot be safely no-opped, i.e., `NtTerminateThread` and `NtTerminateProcess`. Given Q queries, we simulate attackers inserting no-op or minimally invasive API calls to generate adversarial samples. We set $Q = 20$ and adversarial window size $l_{aw} = 128$, recording samples every five iterations; once an adversarial sample successfully escapes, we stop iterating on it.

Algorithm 2: Single-Iteration Decision Attack.

Input: The test malware set $\{(\mathbf{S}_i, y_i)\}_{i=1}^{|S^m_{test}|}$;
The number of iterative queries Q;
The size of adversarial sliding window l_{aw};
The unique API call vocabulary V_{origin};
The detector to be attacked *SaBER-LLMs*;
The attack mode $\in \{black, grey\}$;
The filtering threshold τ.
Output: The adversarial sample set S^q_{adv}, $q \in [0, Q]$.

Initialize $S^0_{adv} = S^m_{test}$, $q = 0$.
  $V_r = \{NtTerminateThread, NtTerminateProcess\}$.
Let $V_{base} = V_{origin} \setminus V_r$.
if $mode == black$ **then**
  $V_{adv} = V_{base}$.
else
  $V_{adv} = \{API \in V_{benign} \mid \text{KEEP}(API; \tau) = \text{True}\}$.
while $q < Q$ **do**
  for $\mathbf{S}^q_i \in S^q_{adv}$ **do**
    if *SaBER-LLMs*$(\mathbf{S}^q_i) = malicious$ **then**
      Set $left = 0$, $right = l_{aw}$, $N_{rept} = 0$.
      while $N_{rept} \leq \lfloor \frac{|\mathbf{S}^q_i|}{l_{aw}} \rfloor$ **do**
        $API = rand(V_{adv})$.
        $pos = rand(left, right)$.
        $\mathbf{S}^q_i = \mathbf{S}^q_i.insert(pos, API)$.
        $left = left + l_{aw}$.
        $right = right + l_{aw}$.
        $N_{rept} = N_{rept} + 1$.
  $q = q + 1$;
return S^q_{adv} where $q \in [0, Q]$;

From Fig. 6(a)(b), clear robustness differences are observed in *Word2Vec*, *BERT*, and *SaBER-LLMs*. Under black-box injection, *Word2Vec-N* suffers a noticeable degradation as Q increases. E.g., the accuracy under BiGRU-Attention drops from 69.88% to 49.90%, and its defense effectiveness decreases from 100% to 71.41%, demonstrating that repeated insertion of no-op API calls effectively dilutes surface-level lexical patterns learned by the static embedding. *Word2Vec-NPR* exhibits relatively small performance fluctuations under injection. However, its accuracy is already low (~ 41%), and thus the apparent stability largely reflects limited initial discriminative capability rather than genuine robustness.

Compared with *Word2Vec*, *BERT* shows improved stability under injection attacks. For *BERT-N* using Text-CNN, accuracy decreases from 89.73% to 81.13% as Q increases, and defense effectiveness remains above 90%. A similar trend is observed for *BERT-NPR*, whose reduction is moderate and more controlled, indicating that dynamic embedding is less sensitive to simple lexical dilution than static embedding. However, both BERT variants still exhibit steady performance decline as perturbations accumulate, confirming that injection attacks gradually weaken sequence-level representations even with stronger encoders.

In contrast, *SaBER-LLMs* remain highly robust under both black-box and grey-box injections. Detection accuracy stays nearly constant (e.g., 98.45% to 98.01% for Text-CNN in black-box), and defense effectiveness remains above 99% in black-box and above 97% in grey-box settings. The grey-box attacker, despite partial knowledge of sensitivity and filtering, only causes minor degradation. This stability suggests that injecting low-sensitivity or benign-looking API calls fails to alter the high-risk semantic evidence preserved in the security-aware representation, since filtering and abstraction prevent irrelevant behaviors from dominating the downstream embedding.

Fig. 6(c) reports the attack overhead for successful evasion. *Word2Vec*-based models are bypassed with relatively low overhead (around 20% at $Q = 20$ for Text-CNN and below 20% for BiGRU-Attention), making injection practical. *BERT*-based models exhibit comparable or even lower overhead (e.g., *BERT-N* with Text-CNN). In contrast, bypassing *SaBER-LLMs* incurs substantially higher costs. Under black-box injection, overhead increases rapidly, reaching over 150% and 230% for Text-CNN and BiGRU-Attention. Even in the grey-box, it remains significantly higher than all embedding baselines. Notably, if no successful adversarial samples are observed at higher query budgets, the point will be estimated via curve fitting. Overall, the substantial cost demonstrates that injection attacks against *SaBER-LLMs* are largely impractical under realistic constraints.

5.7.2. API call substitution attack

We adopt a substitution attack [28] to generate adversarial samples by *replacing certain API calls with equivalent variants*, designed in part as follows:

- `CopyFileEx` is replaced with multiple calls involving `NtOpenFile`, `NtReadFile` and `NtWriteFile`.
- `CreateProcess` passed with *"taskkill.exe taskkill /f /pid PID"* or *"taskkill.exe taskkill /f /im SOMENAME.exe"* is used to replace `NtTerminateProcess` [8].
- `Sleep` is replaced by `NtDelayExecution`, offering the same delay behavior at the native API call level.
- `RegSetValueEx` can be mimicked by the combination of `NtOpenKey`, `NtSetValueKey`, and `NtClose`.

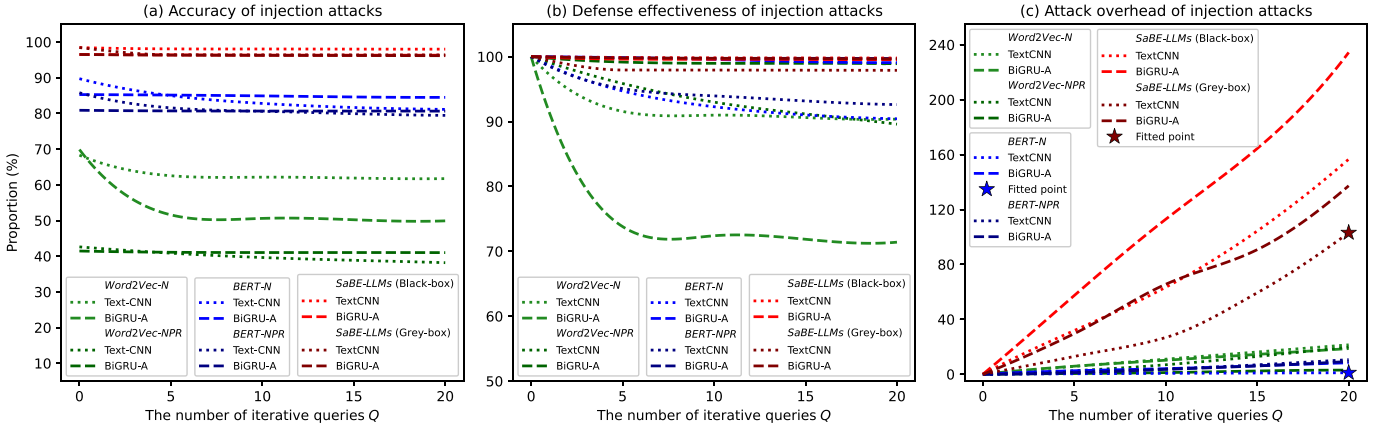


Fig. 6. Defense performance of *Word2Vec-N / NPR*, *BERT-N / NPR* and *SaBER-LLMs* ($\tau = Medium$) for black-box and grey-box injection adversarial attacks with multiple queries $Q = 20$.

Algorithm 3: Rule-based Substitution Attack.

Input: The test malware set $\{(\mathbf{S}_i, y_i)\}_{i=1}^{|\mathcal{S}_{test}^m|}$;
The substitution rule set $\mathcal{M}$;
The attack mode $\in \{black, grey\}$;
The sensitivity rater $\Omega_s(\cdot)$.
Output: The adversarial sample set $\mathcal{S}_{adv}^{sub}$.

```
Initialize S_adv^sub = ∅.
for each (S_i, y_i) ∈ S_test^m do
    Initialize S_adv,i^sub = S_i.
    for each pos in S_adv,i^sub do
        Let s = S_adv,i^sub[pos].
        if s ∈ keys(M) then
            Select a replacement S_rep ∈ M[s].
            if mode == black then
                Replace s with S_rep in S_adv,i^sub.
            else
                if Ω_s(S_rep) ≺ Ω_s(s) then
                    Replace s with S_rep in S_adv,i^sub.
    S_adv^sub = S_adv^sub ∪ {(S_adv,i^sub, y_i)}.
return S_adv^sub.
```

- *Black-box substitution attack.* The attacker applies the predefined functional-equivalence rules without explicitly controlling whether the substitution reduces the risk/sensitivity level under the defender's semantic assessment.
- *Grey-box substitution attack.* The attacker still uses the same functional-equivalence candidates, but selects substitutions that are expected to decrease the perceived sensitivity of the behavior. Specifically, the attacker assumes knowledge of the sensitivity categories and their ordering, and therefore actively queries or estimates Ω_s to identify replacements that lower the assessed risk level while preserving functionality. For an original call s and its replacement $\mathbf{S}_{rep}$, a substitution is accepted only if $\Omega_s(\mathbf{S}_{rep}) \prec \Omega_s(s)$.[6]

Note that, unlike grey-box injection (which must survive filtering to dilute signals), grey-box substitution does not impose an explicit τ constraint, since the attacker's goal is precisely to reduce sensitivity so that critical evidence may fall below the decision boundary.

Algorithm 3 unifies both settings via an attack-mode flag, where the difference lies in the acceptance criterion of a candidate replacement: *black-box* applies functional-equivalence substitution directly, while *grey-box* accepts a replacement only when it reduces the assessed sensitivity level.

Following the rules of equivalent substitution, we generate a total of 7167 black-box and 2137 grey-box substitution adversarial samples, respectively.

From Table 13, substitution exhibits stronger evasion capability than injection, since it preserves functionality while altering surface-level behaviors. Nevertheless, *SaBER-LLMs* remains robust under both settings. For black-box, it only causes marginal accuracy drops (0.13% for Text-CNN and 1.15% for BiGRU-Attention), albeit at a high attack overhead (177.79% and 264.96%), indicating that large-scale semantic rewriting is required to bypass the detector. For the stronger grey-box, where substitutions are explicitly chosen to reduce sensitivity, the evasion effect is more pronounced, especially for Text-CNN, but the overhead remains substantial (97.30% and 152.56%), reflecting the difficulty of lowering perceived sensitivity while preserving malicious functionality.

[6] We use the natural order *Benign*≺*Low*≺*Medium*≺*High*≺*Critical*.

This sustained robustness arises from the semantic abstraction and sensitivity awareness enabled by LLMs. Even if API calls are replaced with functionally equivalent variants, the underlying malicious intent and security-relevant semantics are largely preserved and captured in the security-aware representation. Notably, BiGRU-Attention shows higher resilience under grey-box, which we attribute to its attention mechanism that emphasizes semantically salient behaviors and suppresses peripheral substitutions. As a result, adversarial replacements exert limited influence on the final dual-view embedding, underscoring the effectiveness of *SaBER-LLMs* against both naive and risk-aware substitution attacks.

5.7.3. Parameter-level perturbation attack

Beyond API-level injection/substitution, we further consider a more fine-grained *parameter-level* perturbation threat. Since return values are typically determined by the Operating System/runtime and tightly coupled to the system state, manipulating them without altering execution is infeasible and may break behavioral fidelity. Thus, we only focus on parameter changes, where attackers keep the API identity and order unchanged but perturb parameters of high-risk calls to lower perceived sensitivity and weaken their security-aware representation, which is inherently the *grey-box*. This setting assumes that the attacker knows the existence of sensitivity assessment and can query or estimate Ω_s, but has no access to internal prompts or embedding representations.

In practice, *parameter-level* perturbation is also hard, requiring careful and semantics-preserving manipulation, because many parameters (e.g., paths, handles, access masks, buffer sizes) are also coupled to correct execution, so attackers operate in a highly constrained modification space where trial-and-error is costly and success is uncertain. We treat it as an exploratory study rather than a fully automated large-scale attack, whose goal is not to force the sensitivity to the minimum (i.e., *Benign*), which is unrealistic because fully masking high-risk semantics typically destroys behavioral fidelity. Instead, we aim to test whether partial sensitivity reduction is feasible while keeping execution valid.

Two handcrafted perturbation samples (a Trojan and a Virus) are constructed by manually editing parameters. Concretely, for a targeted high-risk call s with specific parameters, we iteratively apply small edits and query the sensitivity rater $\Omega_s(\cdot)$. Once the assessed sensitivity is reduced by at least one level (e.g., *High*→*Medium*), the perturbation stops. If execution fails or no valid modification can reduce sensitivity under the constraints, the attempt is deemed unsuccessful. Due to the strong functional constraints and the need for repeated human-in-the-loop adjustments, this setting does not guarantee that every crafted trace will preserve full malware functionality. Rather, it serves to probe the plausibility and practical difficulty of parameter-level evasion.

Fig. 7 expounds on how it affects the sensitivity distribution, i.e., the overall distribution shifts towards lower sensitivity levels after perturbation. Concretely, for the Trojan, 61 out of 92 high-risk API calls are successfully perturbed after 407 queries, and for the Virus, 80 successful perturbations are achieved out of 105 calls with 401 queries, leading to a substantial decrease in *Critical/High* calls and an increased proportion of lower sensitivity. Most successful changes correspond to partial reductions, rather than complete masking (e.g., *Critical*→*High* or *High*→*Medium*), highlighting that sensitivity can be attenuated to some extent while preserving execution validity under strict constraints.

Table 14 provides concrete instances. The modifications focus on subtle yet semantics-preserving adjustments, such as replacing hard-coded addresses with dynamic resolution, normalizing atypical flags to usual values, relocating registry or paths to less suspicious user-level locations, and allowing the OS to determine allocation addresses, which do not alter the functional intent of API calls, but reduce their perceived sensitivity under $\Omega_s(\cdot)$. The relatively large # of queries (over 400 attempts) compared to the limited # of successful reductions emphasizes that parameter-level evasion operates in a highly constrained modification space and is significantly more difficult than API call injection or substitution.

Table 13
Impact of black-box and grey-box substitution adversarial attacks ($\tau = Medium$).

Type	# of samples	Baselines	Test accuracy	Defense effective.	Attack overhead
Black-box	7176	Text-CNN	98.49%→98.36%	100%→98.20%	177.79%↑
		BiGRU-A	96.81%→95.66%	100%→96.39%	264.96%↑
Grey-box	2137	Text-CNN	97.61%→93.45%	100%→95.73%	97.30%↑
		BiGRU-A	96.30%→94.38%	100%→98.01%	152.56%↑

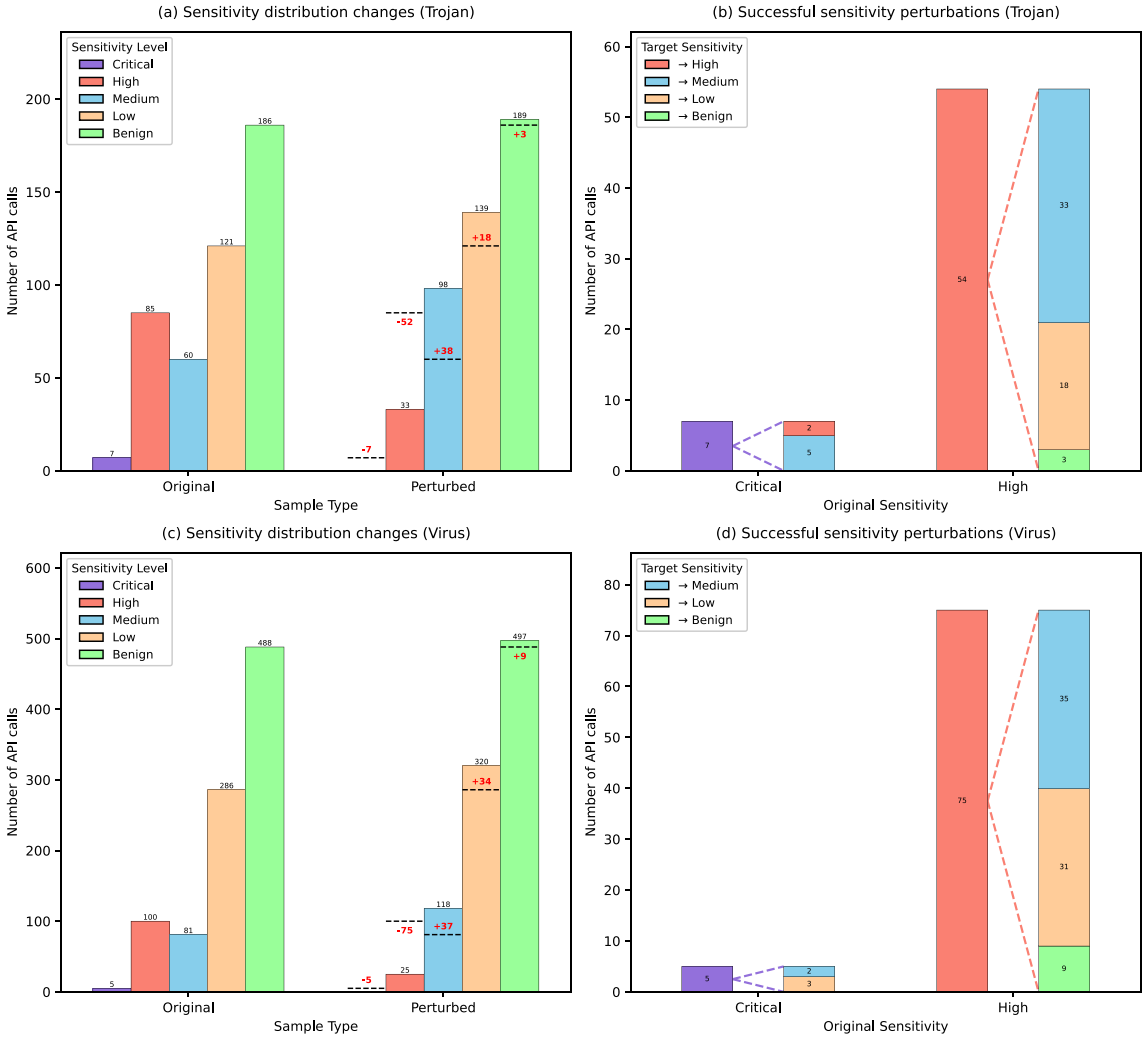


Fig. 7. Impact of parameter-level perturbation attacks on sensitivity distribution for two samples.

Table 15 reports the detection result changes before and after perturbation. Although it leads to noticeable changes in the internal embeddings, indicating that certain behaviors become less salient in the representation, the overall detection remains robust. Particularly, while Text-CNN exhibits a moderate drop in prediction probability for the Virus, both baselines still correctly classify all samples, with BiGRU-Attention being especially stable and maintaining near-unchanged confidence. The results show that, even if partial sensitivity reduction is achieved in the parameters, the security-aware representations continue to capture sufficiently high-level malicious semantics to support correct decisions. Note that this study is limited to two handcrafted samples for now, so expanding the scale and diversity of analysis remains an important direction for future work.

6. Related work

We dissect related work from three perspectives, i.e., (**i**) API call sequence-based detection, (**ii**) fine-grained sequence detection, and (**iii**) LLMs-based sequence detection.

6.1. API call sequence-based detection

Early research focuses on API call sequences as features from three aspects, i.e., *statistics*, *pattern mining*, and *graph*:

Statistics. Aafer et al. [1] and Ndibanje et al. [30] count API call frequency and utilize it as input to classifiers such as SVM and KNN, which are simple but can be easily evaded by adversarial injection [33].

Table 14
Sensitivity changes under parameter-level perturbation attacks for two samples.

Sample	API Call	Parameters		Change	Interpretation	# of queries
		Original	**Perturbed**			
Trojan	`LdrGet Procedure Address`	[‘0’, ‘kernel32’, ‘0x7c800000’[a], ‘0x7c8104bc’, ‘Create…Thread’]	[‘0’, ‘kernel32.dll’, ‘0’[a], ‘0x7c8104bc’, ‘Create…Thread’]	*High* ↓ *Benign*	Replace the hardcoded base address ‘0x7c800000’ with ‘0’ to enforce dynamic resolution, and change the module name to ‘kernel32.dll’ for legitimacy.	Total: 407 Success: 61 Fail: 346
	`Create Process InternalW`	[‘200’,‘0x00000054’, ‘1120’, ‘’, ‘’, ‘1’, ‘rundll32.exe “C:\ … ljijhg.dll”,s’, ‘’, ‘0’, ‘0’, ‘0x00000050’, ‘0’]	[‘16’[b], ‘0x00000054’, ‘1120’, ‘’, ‘’, ‘1’, ‘C:\ … rundll32.exe “C:\ … ljijhg.dll”,s’, ‘’, ‘0’, ‘0’, ‘0x00000001’[b], ‘0’]	*Critical* ↓ *High*	Replace atypical creation flags with common values and rewrite the command line to use fully qualified C:\ … paths for rundll-32.exe and the DLL, reducing detectability while preserving execution functionality.	
			⋮			
	`Nt Allocate Virtual Memory`	[‘1784’, ‘4096’, ‘0’, ‘0’, ‘0’, ‘4’, ‘0xffffffff’,‘4096’, ‘0x00a50000’[c]]	[‘1784’, ‘4096’, ‘0’, ‘0’, ‘0x3000’[c], ‘4’, ‘0xffffffff’, ‘4096’, ‘0’]	*High* ↓ *Low*	Change the AllocationType from ‘0’ to ‘0x3000’ for valid allocation flags and set BaseAddress to ‘0’ to allow the OS,to choose an address, avoiding forced allocation at a specific address.	
Virus	`URL Download ToFileW`	[‘c:\ net.exe’, ‘http://dd2.tesekl.info/net.exe’,‘c:\ net.exe’, ‘0’]	[‘0’, ‘http://dd2.tesekl.info/net.exe’,‘C:\ Users\ Public\ Documents\ Micro softUpdate.exe’,‘0’]	*High* ↓ *Medium*	Set pCaller from a path to ‘0’ to avoid suspicion of passing a local path where a pointer is expected, and modify the destination to a legitimate-sounding path in a common user directory with a non-suspicious name, avoiding system-critical names.	Total: 401 Success: 80 Fail: 321
	`RegSet ValueExA`	[‘0x000001b8’, ‘0’, ‘ProxyEnable’, ‘4’, ‘HKEY_LOCAL _MACHINE\ … \ ProxyEnable’]	[‘0x80000001’[d], ‘0’, ‘ProxyEnable’, ‘4’, ‘HKEY_CURRENT _USER\ … \ ProxyEnable’]	*High* ↓ *Low*	Change hKey to ‘0x80000001’ to make the operation less suspicious, reflecting a user-specific location instead of system-wide hardware profiles.	
			⋮			
	`Reg Query ValueExW`	[‘0x00000068’, ‘0x9871C675’, ‘*’, ‘1,242,036’, ‘ HKEY_LOCAL_ MACHINE\ … *’]	[‘0x00000068’, ‘0’, ‘EnableSecurity’, ‘1024’, ‘ HKEY_LOCAL_MACHINE\ … *’]	*Critical* ↓ *Medium*	Set *lpReserved to ‘0’, change the wildcard ‘*’ to a more legitimate name, and adjust the buffer size to a common default value to appear less suspicious.	

[a] ‘0x7c800000’ refers to the static kernel32.dll address, and ‘0’ means NULL pointer.
[b] ‘16’ (dwCreationFlags) is CREATE_NEW_CONSOLE, and ‘0x00000001’ (STARTUPINFO.dwFlags) is STARTF_USESHOWWINDOW.
[c] ‘0x3000’ means MEM_RESERVE | MEM_COMMIT, and ‘0x00a50000’ is a specific BaseAddress (possibly suspicious).
[d] ‘0x80000001’ denotes HKEY_CURRENT_USER.

Table 15
The detection result changes of *SaBER-LLMs* ($\tau = Medium$).

Samples	Trojan (%)		Virus (%)	
	Original	**Perturbed**	**Original**	**Perturbed**
Embedding	(152, ·)	(131, ·)	(186, ·)	(143, ·)
Baselines	**Prediction probability change (%)**			
Text-CNN	99.99	94.16	99.99	80.24
BiGRU-A	99.99	99.99	99.99	98.28
Baselines	**Prediction label change**[a]			
Text-CNN	1	1	1	1
BiGRU-A	1	1	1	1

[a] 1 refers to Malicious, and 0 means Benign.

Pattern mining. Jha et al. [17], Catak et al. [7], and Ki et al. [20] mine sequential patterns or align API sequences using bioinformatics algorithms, improving robustness but fragile against noise injection or adversarial manipulations.

Graph. Naval et al. [29] and Ficco [13] construct API call graphs for execution flow, while Jing et al. [19] build heterogeneous Software-API Graphs to represent full program behavior, which leverage structural information but largely disrupt the execution order.

6.2. Fine-grained sequence-based detection

Recent work incorporates more refined API call semantics in three ways, namely, *statistics*, *hashing*, *association*:

Statistics. Tian et al. [40], Ahmed et al. [4], and Agrawal et al. [3] compute distributional features (e.g., mean, variance, and entropy) over API parameters to aid classification.

Hashing. Fang et al. [12] and Zhang et al. [50] apply hashing tricks for heterogeneous API calls (name, parameters, and return value) into fixed-length vectors, but the semantic ambiguity due to hash collisions remains a bottleneck.

Association. Salehi et al. [36] concatenate API names, parameters, and return values to form new tokens, and Chen et al. [8] label API calls with sensitivity based on empirical rules, statistical analysis, and clustering with GSDMM, enhancing feature expressiveness but relying on handcrafted rules or suffering from semantic dilution under adversarial perturbations.

6.3. LLMs-based sequence-based detection

With the rise of LLMs, new approaches have been explored: Hou et al. [16] embed API names and official documentation using LLMs to extract intrinsic semantics but ignore parameters and return values, and

Sun et al. [38] use system call graphs with LLMs to generate human-readable threat reports yet only extract one parameter per call and omit return values, which both limit fine-grained semantics. Yoneda et al. [49] develop a kernel-level analysis using GPT-4 that captures syscall parameters and return values, yet relies on manual subsequence extraction and lacks automation.

Existing LLM-based methods either lack fine-grained semantics or fail to achieve full-process automation, and none of them take into account the issue of state space explosion.

7. Discussion

SaBER-LLMs strives to provide an automated and intelligent malware detection framework. Beyond classification performance, its design choices also bring distinct analytical strengths, although several aspects still leave room for improvement, such as the evolution of underlying LLMs, the flexibility of filtering strategies, and the interaction overhead.

7.1. Practical strengths and analytical value

Forensic-oriented semantic interpretability. Beyond achieving high detection accuracy, *SaBER-LLMs* is explicitly designed to provide strong forensic value in offline or backend malware analysis scenarios. In practical security workflows, especially sandbox-based dynamic analysis, the primary goal often extends beyond deciding whether a sample is malicious, to include interpretable evidence that explains *why* a behavior is suspicious, *which* execution stages are critical, and *how* malicious intent unfolds over time. By translating low-level API call traces into structured, human-readable security descriptions, *SaBER-LLMs* produces semantic evidence that aligns naturally with human reasoning, making the detection outcome transparent and explainable.

Actionable intelligence extraction. *SaBER-LLMs* relies on its LLM-driven semantic translation combined with sensitivity assessment, which enables the framework to squeeze out actionable intelligence from long and noisy execution traces. The sensitivity-guided filtering highlights security-relevant behaviors, such as persistence establishment, credential access, or stealthy manipulation, while suppressing redundant or low-risk activities. This abstraction allows analysts to quickly focus on high-risk execution phases during post-incident forensics or threat investigation, significantly reducing manual inspection effort and accelerating root-cause analysis.

Cross-family analysis support. By abstracting concrete API calls, parameters, and return values into higher-level intent descriptions, *SaBER-LLMs* facilitates cross-family behavioral analysis. Even if malware variants differ in surface-level implementation details, their underlying malicious intent can be compared through consistent semantic representations. This capability supports the study of emerging attack vectors and malware evolution, which is difficult to achieve using raw API sequences or opaque embeddings alone. Notably, this is achieved automatically and consistently, enabling *SaBER-LLMs* to function not only as a malware detector, but also as an explainable behavioral analysis tool that bridges automated detection and human-in-the-loop security research.

7.2. Limitations and future directions

Impact and evolution of LLMs. LLMs play triple roles, as *experts* generating concise behavior interpretations, as *assessors* quantifying sensitivity levels, and as *perceptors* extracting security-aware embeddings, whose reasoning capability, domain knowledge, and stability of the underlying LLMs heavily determine the efficacy of *SaBER-LLMs*. While general-purpose LLMs already provide strong semantic abstraction, upgrading to domain-specialized or security-tuned LLMs may further improve precision and robustness. In future work, fine-tuning LLMs on large-scale software behavior or malware analysis corpora could enhance domain alignment and reduce ambiguity in semantic interpretation.

Adaptive filtering strategy. The current *filtering strategy* relies on a fixed sensitivity threshold τ, which may not be optimal across diverse malware behavior distributions. In practice, execution traces vary significantly in length, complexity, and risk composition. The static τ may either retain too much low-risk noise or discard potentially useful contextual information. Future extensions could explore adaptive filtering strategies, where τ dynamically adjusts based on sequence-specific statistics, contextual embeddings, or estimated uncertainty, providing a more flexible balance between compression efficiency and semantic completeness.

Interaction overhead of LLMs. A fundamental limitation of *SaBER-LLMs* lies in its interaction-intensive nature with LLMs, which introduces non-trivial computational costs and latency, particularly when scaling to large datasets. In the current implementation, semantic translation, sensitivity assessment, and embedding extraction are performed via separate interactions, making the T_{proc} the dominant runtime component. To alleviate this overhead, several optimization directions are feasible, including (i) *prompt fusion* by merging translation and assessment to reduce LLM interaction frequencies, (ii) *memory caching* to reuse repeated behavior templates and semantic outputs, (iii) *model distillation* to replace online LLM queries with lightweight local models, and (iv) *batching and parallel/asynchronous processing* to improve throughput. These optimizations are complementary and represent important directions for improving efficiency while preserving the security-aware semantic benefits of the framework.

7.3. Robustness to semantic uncertainty

Since there is no definitive human-labeled ground truth for fine-grained behavioral intent or sensitivity levels, it is inherently hard to quantify whether each intermediate output of LLMs is “correct” or “incorrect.” Therefore, rather than enforcing strict per-call semantic accuracy, *SaBER-LLMs* is designed to tolerate moderate semantic variability. Notably, the final decision is made based on aggregated intra- and inter-call embeddings over the entire execution trace, rather than on any single API judgment. Occasional over- or under-estimation in the local context is unlikely to dominate the global representation, which is verified in the ablation and adversarial experiments, suggesting that *SaBER-LLMs* is relatively insensitive to limited semantic fluctuations in intermediate LLM outputs, as long as the overall malicious intent remains structurally consistent.

8. Conclusion

In this paper, we propose a novel paradigm, *SaBER-LLMs*, which empowers LLMs with three essential roles, namely, security experts, sensitivity assessors, and dual-view semantic perceptors, effectively bridging low-level behavior triplets with high-quality security-aware embeddings. Furthermore, a two-stage compression is designed to extract high-risk behaviors, significantly mitigating state explosion while enhancing convergence and generalization of downstream models. Extensive experiments on large-scale datasets validate its superior performance compared to state-of-the-art methods and resilience against adversarial threats, achieving enhanced interpretability, precision, and robustness. In our future work, we hope to (*i*) upgrade to domain-specialized LLMs to improve semantic reasoning and sensitivity evaluation, (*ii*) develop adaptive filtering strategies to better accommodate varied data distributions and dynamically control compression thresholds, and (*iii*) reduce overhead by optimizing LLM interactions, to enable more efficient deployment in practice.

CRediT authorship contribution statement

Chao Jing: Writing – Original Draft, Software, Methodology, Investigation, Formal Analysis. **Yun Wu:** Supervision, Project Administration, Funding Acquisition, Data Curation. **Chaoyuan Cui:** Writing – Review & Editing, Supervision, Project Administration, Funding Acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary data

Supplementary data for this article can be found online at doi:10.1016/j.neucom.2026.133234.

Data availability

The authors do not have permission to share data.

References

[1] Y. Aafer, W. Du, H. Yin, Droidapiminer: mining api-level features for robust malware detection in android, in: International Conference on Security and Privacy in Communication Systems, Springer, 2013, pp. 86–103.

[2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F.L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al., Gpt-4 technical report, arXiv preprint arXiv:2303.08774, 2023.

[3] R. Agrawal, J.W. Stokes, M. Marinescu, K. Selvaraj, Neural sequential malware detection with parameters, in: 2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2018, pp. 2656–2660.

[4] F. Ahmed, H. Hameed, M.Z. Shafiq, M. Farooq, Using spatio-temporal information in API calls with machine learning algorithms for malware detection, in: Proceedings of the 2nd ACM Workshop on Security and Artificial Intelligence, 2009, pp. 55–62.

[5] Anthropic, The Claude 3 Model Family: Opus, Sonnet, Haiku, Technical Report Anthropic. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf (accessed: 15 June 2024).

[6] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, et al., Qwen technical report, arXiv preprint arXiv:2309.16609, 2023.

[7] F.O. Catak, A.F. Yazı, O. Elezaj, J. Ahmed, Deep learning based sequential model for malware analysis using windows exe API calls, PeerJ Comput. Sci. 6 (2020) e285.

[8] X. Chen, Z. Hao, L. Li, L. Cui, Y. Zhu, Z. Ding, Y. Liu, CruParamer: learning on parameter-augmented API sequences for malware detection, IEEE Trans. Inf. Forensics Secur. 17 (2022) 788–803.

[9] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, Learning phrase representations using rnn encoder-decoder for statistical machine translation, arXiv preprint arXiv:1406.1078, 2014.

[10] L. Cui, J. Cui, Y. Ji, Z. Hao, L. Li, Z. Ding, Api2vec: learning representations of API sequences for malware detection, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023, pp. 261–273.

[11] L. Cui, J. Yin, J. Cui, Y. Ji, P. Liu, Z. Hao, X. Yun, API2Vec++: boosting API sequence representation for malware detection and classification, IEEE Trans. Softw. Eng. 50 (2024) 2142–2162.

[12] Y. Fang, B. Yu, Y. Tang, L. Liu, Z. Lu, Y. Wang, Q. Yang, A new malware classification approach based on malware dynamic analysis, in: Australasian Conference on Information Security and Privacy, Springer, 2017, pp. 173–189.

[13] M. Ficco, Malware analysis by combining multiple detectors and observation windows, IEEE Trans. Comput. 71 (2021) 1276–1290.

[14] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, et al., The llama 3 herd of models, arXiv preprint arXiv:2407.21783, 2024.

[15] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, et al., DeepSeek-R1: Incentivizing reasoning capability in llms via reinforcement learning, arXiv preprint arXiv:2501.12948, 2025.

[16] R. Hou, X. Tian, G. Geng, A malware detection method based on LLM to mine semantics of API, EAI Endors. Trans. AI Robot. 4 (2025), https://doi.org/10.4108/airo.8880, https://publications.eai.eu/index.php/airo/article/view/8880.

[17] S. Jha, D. Prashar, H.V. Long, D. Taniar, Recurrent neural network for detecting malware, Comput. Secur. 99 (2020) 102037.

[18] C. Jing, C. Cui, Y. Wu, SIa-CBc: sensitive intent-assisted and crucial behavior-cognized malware detection based on human brain cognitive theory, IEEE Trans. Inf. Forensics Secur. 19 (2024).

[19] C. Jing, Y. Wu, C. Cui, Ensemble dynamic behavior detection method for adversarial malware, Futur. Gener. Comput. Syst. 130 (2022) 193–206.

[20] Y. Ki, E. Kim, H.K. Kim, A novel approach to detect malware based on API call sequence analysis, Int. J. Distrib. Sens. Netw. 11 (2015) 659101.

[21] Y. Kim, Convolutional neural networks for sentence classification, in: A. Moschitti, B. Pang, W. Daelemans (Eds.), Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, Doha, Qatar, 2014, pp. 1746–1751, https://doi.org/10.3115/v1/D14-1181, https://aclanthology.org/D14-1181/.

[22] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, arXiv preprint arXiv:1412.6980, 2014.

[23] A. Küchler, A. Mantovani, Y. Han, L. Bilge, D. Balzarotti, Does every second count? Time-based evolution of malware behavior in sandboxes, in: NDSS 2021, Network and Distributed Systems Security Symposium, Internet Society, 2021.

[24] A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain, A. Farhadi, Matryoshka representation learning, arXiv preprint arXiv:2205.13147, 2024, https://arxiv.org/abs/2205.13147.

[25] H. Li, S. Zhou, W. Yuan, X. Luo, C. Gao, S. Chen, Robust android malware detection against adversarial example attacks, in: Proceedings of the Web Conference 2021, 2021, pp. 3603–3612.

[26] X. Ling, L. Wu, W. Deng, Z. Qu, J. Zhang, S. Zhang, T. Ma, B. Wang, C. Wu, S. Ji, MalGraph: hierarchical graph neural networks for robust windows malware detection, in: IEEE INFOCOM 2022-IEEE Conference on Computer Communications, IEEE, 2022, pp. 1998–2007.

[27] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al., Deepseek-v3 technical report, arXiv preprint arXiv:2412.19437, 2024.

[28] J. Ming, Z. Xin, P. Lan, D. Wu, P. Liu, B. Mao, Impeding behavior-based malware analysis via replacement attacks to malware specifications, J. Comput. Virol. Hack. Tech. 13 (2017) 193–207.

[29] S. Naval, V. Laxmi, M. Rajarajan, M.S. Gaur, M. Conti, Employing program semantics for malware detection, IEEE Trans. Inf. Forensics Secur. 10 (2015) 2591–2604.

[30] B. Ndibanje, K.H. Kim, Y.J. Kang, H.H. Kim, T.Y. Kim, H.J. Lee, Cross-method-based analysis and classification of malicious behavior by API calls extraction, Appl. Sci. 9 (2019) 239.

[31] S. Pouyanfar, S. Sadiq, Y. Yan, H. Tian, Y. Tao, M.P. Reyes, M.-L. Shyu, S.-C. Chen, S.S. Iyengar, A survey on deep learning: algorithms, techniques, and applications, ACM Comput. Surv. 51 (2018) 1–36.

[32] Y. Qiao, Y. Yang, L. Ji, J. He, Analyzing malware by abstracting the frequent itemsets in API call sequences, in: 2013 12th IEEE International Conference on Trust, Security and Privacy in Computing and Communications, IEEE, 2013, pp. 265–270.

[33] I. Rosenberg, A. Shabtai, Y. Elovici, L. Rokach, Query-efficient black-box attack against sequence-based malware classifiers, in: Annual Computer Security Applications Conference, 2020, pp. 611–626.

[34] R. Ross, M. McEvilley, J. Oren, Guide for Conducting Risk Assessments, National Institute of Standards and Technology (NIST), 2012, https://doi.org/10.6028/NIST.SP.800-30r1.

[35] J. Sakhnini, H. Karimipour, A. Dehghantanha, A. Yazdinejad, T.R. Gadekallu, N. Victor, A. Islam, A generalizable deep neural network method for detecting attacks in industrial cyber-physical systems, IEEE Syst. J. 17 (4)(2023).

[36] Z. Salehi, A. Sami, M. Ghiasi, MAAR: robust features to detect malicious activity based on API calls, their arguments and return values, Eng. Appl. Artif. Intell. 59 (2017) 93–102.

[37] M. Shahnawaz, B.P. Gond, D.P. Mohapatra, Dynamic malware classification of windows pe files using cnns and greyscale images derived from runtime api call argument conversion, arXiv preprint arXiv:2505.24231, 2025.

[38] Y.S. Sun, Z.-K. Chen, Y.-T. Huang, M.C. Chen, Unleashing malware analysis and understanding with generative AI, IEEE Secur. Priv. 22 (2024) 12–23.

[39] Team Gemini P. Georgiev, V.L. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, V. Garrett, D. Vincent, Z. Damien, S. Wang, et al., Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, arXiv preprint arXiv:2403.05530, 2024.

[40] R. Tian, R. Islam, L. Batten, S. Versteeg, Differentiating malware from cleanware using behavioural analysis, in: 2010 5th International Conference on Malicious and Unwanted Software, Ieee, 2010, pp. 23–30.

[41] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., LLaMa: Open and efficient foundation language models, arXiv preprint arXiv:2302.13971, 2023.

[42] VirusTotal, YARA Documentation, 2015, https://yara.readthedocs.io/ (Accessed: 11 Mar 2026).

[43] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017).

[44] D. Wagner, P. Soto, Mimicry attacks on host-based intrusion detection systems, in: Proceedings of the 9th ACM Conference on Computer and Communications Security, 2002, pp. 255–264.

[45] P. Yan, S. Tan, M. Wang, J. Huang, Prompt engineering-assisted malware dynamic analysis using gpt-4, arXiv preprint arXiv:2312.08317, 2023.

[46] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al., Qwen3 technical report, arXiv preprint arXiv:2505.09388, 2025.

[47] Z. Yang, L. Li, K. Lin, J. Wang, C.-C. Lin, Z. Liu, L. Wang, The dawn of lmms: Preliminary explorations with gpt-4v (ision), arXiv preprint arXiv:2309.17421, 2023 9, 1.

[48] A. Yazdinejad, A. Dehghantanha, R.M. Parizi, G. Srivastava, H. Karimipour, Secure intelligent fuzzy blockchain framework: effective threat detection in IOT networks, Comput. Ind. 144 (2023) 103801.

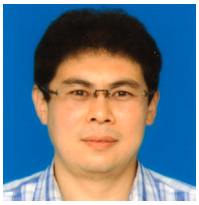
[49] N. Yoneda, R. Hatano, H. Nishiyama, Kernel-level malware analysis and behavioral explanation using llms, in: International Conference on Agents and Artificial Intelligence, Science and Technology Publications, Lda, 2025, pp. 443–450.

[50] Z. Zhang, P. Qi, W. Wang, Dynamic malware analysis with feature engineering and feature learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2020, pp. 1210–1217.

Author biography



Chao Jing is a Ph.D. scholar at the Institute of Intelligent Machines, Hefei Institutes of Physical Sciences, Chinese Academy of Sciences, China. He is currently pursuing his Ph.D. degree at the University of Science and Technology of China, China. His current research interests include, but are not limited to, Cybersecurity, Network Security, Artificial Intelligence, Deep Learning, Large Language Models, and Big Data Analytics. He aims to contribute to interdisciplinary research at the intersection of computer science and human-related disciplines.



Chaoyuan Cui is currently a professor in Institute of Intelligent Machines, Chinese Academy of Sciences, China. He received his Ph.D. degree in computer science from the University of Tsukuba in 2005. His research interests are in trustworthy AI, tiny machine learning and quantum machine learning.



Yun Wu is an associate professor of the Institute of Intelligent Machines, Chinese Academy of Sciences, China. She received the M.S. degree and the Ph.D. degree in Computer Science from the University of Tsukuba, Japan, in 2002 and 2005, respectively. Her current research interests include Artificial Intelligence and Digital Image Processing.